

UNIVERSITY OF CAPE TOWN



Comparing Louvain and Leiden community detection performance across network topologies

Student(s):

Laylaa Jorge Abrahams
Khethani Muger

Supervisor:

Mr Stefan S Britz

**Honours research project
in the**

DEPARTMENT OF STATISTICAL SCIENCES

March 21, 2026

Abstract

Comparing Louvain and Leiden community detection performance across network topologies

This study investigates and compares the performance of two widely used community detection algorithms – Louvain and Leiden – across three different network topologies: random (Erdős–Rényi), small-world (Watts–Strogatz), and scale-free networks (Barabási–Albert), while systematically varying network size, mean degree, and the resolution parameter specified in the algorithms. The quality and robustness of detected communities are evaluated via three quality functions, namely modularity, conductance, and stability.

Results show that network topology had a stronger influence on community detection performance than algorithmic differences. In random networks, both the Louvain and Leiden algorithms produced low modularity, high conductance, and low stability, reflecting the absence of meaningful community structure. Small-world networks showed higher modularity, lower conductance, and greater stability, with the Leiden algorithm performing slightly better, particularly in smaller and sparser networks. Scale-free networks displayed moderate modularity, conductance and stability, as highly connected hubs reduced community separability. Increasing the resolution parameter generally decreased modularity while increasing conductance and stability, indicating smaller and more refined communities with weaker separation.

Overall, both algorithms exhibited similar trends across all network types, with Leiden offering marginal improvements likely due to its refinement step that produces more cohesive partitions. These results demonstrate that community quality metrics are primarily determined by network structure, while algorithmic differences play a minor but consistent role in enhancing detection performance.

Key words:

Community detection, Louvain, Leiden, Random networks, Small-world networks, Scale-free networks, modularity, conductance, stability

Contents

Abstract	i
1 Introduction	1
1.1 Network Topologies	2
1.2 Community Detection	2
1.2.1 Community Detection Quality Functions	3
1.3 Research Objectives	4
1.4 Dissertation Outline	4
2 Literature Review	6
2.1 Network science	6
2.2 Network Analysis Theory and Application	7
2.3 Network topologies	9
2.3.1 Random networks (Erdős–Rényi)	9
2.3.2 Small-world networks (Watts–Strogatz)	9
2.3.3 Scale-free networks (Barabási–Albert)	10
2.4 Community detection	11
2.4.1 The Louvain algorithm	13
2.4.2 The Leiden Algorithm	15
3 Methodology	18
3.1 Network topologies	18
3.2 Community Detection	20
3.3 Assessing community detection algorithms	21
3.3.1 Modularity	22
3.3.2 Conductance	22
3.3.3 Stability	23
3.4 Reproducibility and Experimental Design	24
4 Results and Discussion	26
4.1 Varying network size and mean degree	26
4.1.1 Random networks	26
4.1.2 Small-world networks	29
4.1.3 Scale-free networks	31

4.2	Varying size and resolution at constant mean degree	34
4.2.1	Random networks	34
4.2.2	Small-world networks	35
4.2.3	Scale-free networks	36
4.3	Discussion	37
5	Conclusion	40
5.1	Challenges and Limitations	41
5.2	Future Work and Recommendations	41
A	Appendix	43
A.1	Random networks at a fixed mean degree	43
A.2	Small-world networks at a fixed mean degree	51
A.3	Scale-free networks at a fixed mean degree	60
	Bibliography	70

List of Figures

2.1	Generated random network before and after community detection	11
2.2	Schematic illustration of the Louvain algorithm's two main phases	14
2.3	Graphical illustration of the Louvain's badly connected communities	15
2.4	Schematic illustration of the Leiden algorithms	17
3.1	Example of a random network produced with 16 nodes, a resolution of 1 and a mean degree of 4.	19
3.2	Small-world network produced with 16 nodes, a mean degree of 4, a resolution of 1 and a rewiring probability of 0.10.	20
3.3	Scale-free network produced with 64 nodes, a resolution of 1 and a mean degree of 4 specified.	20
3.4	A random network before community detection and after Louvain and Leiden algorithms are run on the networks.	21
3.5	A small-world network before community detection and after Louvain and Leiden algorithms are run on the network.	21
3.6	A scale-free network before community detection and after Louvain and Leiden algorithms are run on the network.	21
4.1	Average modularity results for random networks	27
4.2	Average conductance results for random networks	28
4.3	Average stability results for random networks of different sizes.	28
4.4	Average modularity results for small-world networks	29
4.5	Average conductance results for small-world networks	30
4.6	Average stability results for small-world networks	31
4.7	Average modularity results for scale-free networks	31
4.8	Average conductance results for scale-free networks	32
4.9	Average stability results for scale-free networks	33
A.1	Average modularity results for random networks at a mean degree of four. . .	43
A.2	Average modularity results for random networks at a mean degree of six. . . .	44
A.3	Average modularity results for random networks at a mean degree of eight. .	44
A.4	Average modularity results for random networks at a mean degree of ten. . .	45
A.5	Average conductance results for random networks at a mean degree of four. .	46
A.6	Average conductance results for random networks at a mean degree of six. . .	46

A.7	Average conductance results for random networks at a mean degree of eight. .	47
A.8	Average conductance results for random networks at a mean degree of ten. . .	47
A.9	Average stability results for random networks at a mean degree of four. . . .	48
A.10	Average stability results for random networks at a mean degree of six.	49
A.11	Average stability results for random networks at a mean degree of eight. . . .	50
A.12	Average stability results for random networks at a mean degree of ten.	51
A.13	Average modularity results for small-world networks at a mean degree of four.	52
A.14	Average modularity results for small-world networks at a mean degree of six.	52
A.15	Average modularity results for small-world networks at a mean degree of eight.	53
A.16	Average modularity results for small-world networks at a mean degree of ten.	53
A.17	Average conductance results for small-world networks at a mean degree of four.	54
A.18	Average conductance results for small-world networks at a mean degree of six.	55
A.19	Average conductance results for small-world networks at a mean degree of eight.	55
A.20	Average conductance results for small-world networks at a mean degree of ten.	56
A.21	Average stability results for small-world networks at a mean degree of four. .	57
A.22	Average stability results for small-world networks at a mean degree of six. . .	58
A.23	Average stability results for small-world networks at a mean degree of eight. .	59
A.24	Average stability results for small-world networks at a mean degree of ten. . .	60
A.25	Average modularity results for scale-free networks at a mean degree of four. .	61
A.26	Average modularity results for scale-free networks at a mean degree of six. . .	61
A.27	Average modularity results for scale-free networks at a mean degree of eight. .	62
A.28	Average modularity results for scale-free networks at a mean degree of ten. . .	62
A.29	Average conductance results for scale-free networks at a mean degree of four.	63
A.30	Average conductance results for scale-free networks at a mean degree of six. .	64
A.31	Average conductance results for scale-free networks at a mean degree of eight.	64
A.32	Average conductance results for scale-free networks at a mean degree of ten. .	65
A.33	Average stability results for scale-free networks at a mean degree of four. . . .	66
A.34	Average stability results for scale-free networks at a mean degree of six. . . .	67
A.35	Average stability results for scale-free networks at a mean degree of eight. . .	68
A.36	Average stability results for scale-free networks at a mean degree of ten. . . .	69

List of Tables

3.1	Summary of the experimental design for community detection evaluation. . .	25
-----	--	----

Chapter 1

Introduction

Network analysis has become an important approach for studying complex systems. It is used across many disciplines, including sociology, biology, and computer science. According to Brandes (2005), network graphs provide a natural mathematical framework for modeling networks. A network is a set of nodes and edges, in which the edges connect nodes. Hevey (2018) stated that networks are graphical representations of relationships (edges) between variables (nodes). Networks can be directed or undirected. In undirected networks, the relationships between connected nodes are symmetrical or reciprocal, whereas in directed networks, the relationships between connected nodes are asymmetrical (Yang et al., 2014).

Many aspects of the real world can be represented by networks. Road networks with cities as nodes and roads as edges, social media with accounts as nodes and following or “friendship” as edges and brain connectomes¹ with neurons as nodes and synapses as edges are just three of the many examples (Rossi and Ahmed, 2015). Analysing these structures helps identify patterns of interaction and groups of closely connected individuals.

Networks can be studied at various levels of resolution: the network level, the community level, and the node level. This study focuses on the community level. Communities are defined as groups of nodes that are densely connected internally but weakly connected to the rest of the network (Fenn et al., 2009).

This project aims to compare the performance of the Louvain and Leiden community detection algorithms using three quality functions: modularity, conductance, and stability. Section 3.3 explains how each metric is calculated and interpreted. The comparison is performed across three established network topologies: Erdős–Rényi (random), Watts–Strogatz (small-world), and Barabási–Albert (scale-free) networks.

¹Connectomes are comprehensive maps of neural connections within an organism’s nervous system. They represent the intricate web of neurons and synapses that underpin all neural activity and cognition.

1.1 Network Topologies

Understanding the underlying structure of a network is essential for analysing its behaviour and for assessing the performance of community detection algorithms (see section 1.2). Different network topologies exhibit distinct patterns of connectivity and clustering, which influence how communities emerge and interact. This section provides an overview of the network structures considered in this study, highlighting their defining properties.

A random network begins with n isolated nodes, with edges added at random (Bollobás, 2001). The random network model, introduced by Erdős and Rényi (1959), forms networks by connecting pairs of nodes independently with a fixed probability. The Erdős–Rényi framework includes two main variants: the Gilbert model, $G(n, p)$, in which each pair of nodes connects independently with probability p , and the Erdős–Rényi model, $G(n, M)$, where all graphs with exactly M edges have equal probability (Shahraeini, 2024).

Small-world networks, introduced by Watts and Strogatz (1998), combine the high clustering typical of regular lattices with the short path lengths found in random networks. In such networks, the average distance between any two nodes remains small, even as the network grows. Small-world properties have been observed in real systems, including social networks of film actors, and technological infrastructures such as the Western United States power grid (Watts and Strogatz, 1998).

Lastly, Scale-free networks differ by exhibiting heavy-tailed degree distributions that follow a power law. Barabási and Albert (1999) introduced the concept of preferential attachment, where new nodes are more likely to link to already well-connected nodes. This process produces highly connected hubs that dominate the network structure. Scale-free properties appear in many real systems, including the Internet, biological interaction networks, and communication networks (Lynn et al., 2024).

Together, these models represent key structural patterns for studying network behaviour and testing the robustness of community detection algorithms. Thus, the next section examines community detection algorithms.

1.2 Community Detection

Community detection is the process of discovering groups in a network where individuals' group memberships are not explicitly given (Natarajan, 2016). Detecting these substructures helps explain how network topology influences interaction patterns.

A wide variety of community detection algorithms have been developed to uncover the structure of complex networks. These algorithms aim to partition nodes into groups such that the nodes within the same group are more densely connected with one another than the rest of the network. Numerous methods have been proposed in the literature, including hierarchical

clustering, statistical inference, spectral clustering, and modularity optimization (Anandkumar et al., 2014). However, this project focuses on modularity-based algorithms, namely the Louvain and Leiden community detection algorithms.²

The Louvain algorithm optimizes a quality function, such as modularity (see next Section), through two main phases: the local movement of nodes and the aggregation of the network (Traag et al., 2019). The detailed mechanism of the Louvain algorithm is described in Section 2.4.1. Although efficient, the Louvain algorithm often produces poorly connected communities, where some nodes within a community are not directly connected to others. This weak connectivity limits the accuracy of detected structures.

The Leiden algorithm addresses this limitation by introducing a refinement phase between the local movement and aggregation steps (see Section 2.4.2 for a detailed mechanism) (Traag et al., 2019). This phase checks whether each community is internally well-connected. A well-connected community is one in which the nodes inside the community are strongly linked to each other – that is, most nodes can reach one another through short paths without needing to go outside the community. If not, the community is divided into smaller, well-connected subgroups. The aggregation phase then constructs a new network based on this refined partition. This improvement ensures that all resulting communities are internally cohesive, leading to more reliable detection of network structure. The following section provides information on the quality functions used to evaluate the performance of each community detection algorithm.

1.2.1 Community Detection Quality Functions

To assess the quality of communities and the performance of the Louvain and Leiden algorithms, this study employs three quality functions: modularity, conductance, and stability. Using multiple metrics provides a more complete evaluation of algorithm performance across networks with different topologies.

Modularity is the most widely used quality function for community detection. Unlike most partitioning methods, it does not require specifying the number of communities in advance (Delvenne, Yaliraki, et al., 2010). Modularity measures the difference between the actual number of edges in a community and the expected number of edges (Traag et al., 2019). In simple terms, modularity tries to measure how densely nodes are connected within a community.

Conductance evaluates how well a community is separated from the rest of the network. Conductance is the fraction of edges leaving the community relative to the total degree of nodes within the community. While modularity highlights how strongly nodes are connected

²The Louvain algorithm was developed by Vincent Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre at the Université catholique de Louvain in Belgium (Blondel et al., 2008). The Leiden algorithm was later developed by Vincent Traag, Ludo Waltman, and Nees Jan van Eck at Leiden University in the Netherlands (Traag et al., 2019). Both algorithms are named after their respective universities.

within a community, conductance focuses on how few connections it has to the rest of the network, providing a complementary view of community structure.

Lastly, stability measures the reliability of community detection by evaluating the similarity of partitions across multiple runs. This metric is particularly important because both the Louvain and Leiden algorithms are stochastic in nature, meaning they may produce different community assignments in repeated executions. Assessing stability, therefore, provides insight into the consistency of the detected communities.

1.3 Research Objectives

In this project, we aim to analyse the performance of community detection algorithms across different simulated network topologies. The objectives are as follows:

1. Simulate three network topologies:
 - Erdős–Rényi (Random) networks
 - Barabási–Albert (Scale-free) networks
 - Watts–Strogatz (Small-world) networks
2. Apply community detection algorithms to the generated networks:
 - Louvain algorithm
 - Leiden algorithm
3. Evaluate algorithm performance using the following quality functions:
 - Modularity
 - Conductance
 - Stability
4. Investigate how algorithm performance varies as key network characteristics are systematically varied:
 - Graph sizes: 16, 32, 64, 128, 256, 512
 - Mean degree: 4, 6, 8, 10
 - Resolution parameters: 0.5 to 5.0, in increments of 0.5

1.4 Dissertation Outline

Having introduced the research area and objectives in this chapter, the next chapter reviews relevant literature, covering the theory of network science, applications of network analysis, and community detection algorithms used on the network topologies. The contributions of previous studies are summarized.

Chapter three details the methodology. It describes the mechanisms of each network topology, the operation of the community detection algorithms, and the quality functions. The chapter also outlines the experimental design employed in this study.

The results are presented and discussed in Chapter 4, illustrating how variations in network size and mean degree affect performance metrics. In this chapter, we also investigate the impact of changing network size and the resolution parameter while keeping the mean degree constant. Results for when all mean degrees are fixed are interpreted in Chapter 4 and the plots are found in the Appendix.

The final chapter presents conclusions. It summarizes the research objectives and key findings, discusses limitations and challenges encountered during the study, and provides recommendations for future research.

Chapter 2

Literature Review

This chapter explores the evolution and application of network science, with an emphasis on network topologies and community detection algorithms. It introduces key network models – the Erdős–Rényi (random), Watts–Strogatz (small-world), and Barabási–Albert (scale-free) frameworks – and examines how these structures affect the identification of communities within networks. Particular focus is given to the Louvain and Leiden algorithms, two prominent approaches to community detection, by drawing on the foundational contributions of Blondel et al. (2008) and Traag et al. (2019). The discussion critically reviews their theoretical foundations, computational methodologies, and comparative performance across different network topologies, establishing the basis for evaluating the effectiveness of the Leiden algorithm relative to the Louvain algorithm in identifying modular structures.

2.1 Network science

Network science is a highly interdisciplinary field that seeks to develop theoretical and practical approaches for understanding both natural and man-made networks (Börner et al., 2007). Its roots date back to 1736, when Leonhard Euler solved the Bridges of Königsberg problem, marking the birth of graph theory and demonstrating its potential to address real-world connectivity challenges (Lewis, 2009).

According to Barabási (2013), network science aims to understand the networks emerging in nature, technology, and society using a unified set of principles and analytical tools. For much of the subsequent centuries, graph theory remained a niche branch of pure mathematics. It experienced a revival in the mid-20th century through the pioneering work of Paul Erdős, whose studies on random graphs both advanced discrete mathematics and laid the foundation for the Erdős–Rényi (ER) random graph model – a benchmark that continues to play a central role in network analysis (Erdős and Rényi, 1959). By the late 1990s, these mathematical foundations had been revitalized into what is now recognized as network science, a multidisciplinary field with applications across physics, biology, computer science, and the social sciences (Lewis, 2009).

An article by Brandes et al. (2013) advances three core claims about the nature of network science. First, network science is fundamentally the study of network models, but these models are context-dependent and unlikely to generalize across domains. Rather than assuming a universal “Grand Unified Network Theory,” network science should remain sensitive to the phenomena being modeled and avoid mislabeling domain-specific applications (e.g., calling all network studies “social network analysis”). Second, Brandes et al. (2013) contended that network theory involves both the representation of phenomena as networks and the theoretical frameworks explaining them. Viewing a system as a network is not a mere analytical tool but a distinct theoretical commitment that emphasizes relationships and structural patterns as fundamental to understanding complex systems. Crucially, abstracting a phenomenon as a network is itself a theoretical step that enables new kinds of inference, beyond mere aggregation of individual elements. Third, network science should maintain a consistently empirical orientation, with its value assessed in comparison to alternative representations.

Overall, the literature positions network science as a unifying framework for analysing complex, interconnected systems across multiple disciplines. Its strength lies not only in methodological innovations, such as models and analytical tools, but also in the theoretical commitment to representing phenomena as networks. This dual foundation has enabled wide-ranging applications in social, ecological, biological, and technological domains, offering valuable insights into the structure, function, and dynamics of networks.

2.2 Network Analysis Theory and Application

The theoretical foundations of network analysis can be traced to the early work of Wellman (1983), who articulated network analysis as a distinct theoretical framework within the social sciences. Brandes et al. (2013) earlier claim, however, emphasizes that the study of network models is context-dependent and cautions against assuming universal applicability. Moving beyond mere methodological tools, Wellman (1983) emphasized that the essence of network analysis lies in examining social structures through patterns of relations rather than attributes of individual actors. Similarly, Emirbayer and Goodwin (1994) highlighted the theoretical assumptions of network analysis and demonstrated its application to the study of social structures, providing evidence that early uses of network analysis were primarily focused on understanding social relational patterns.

Building on these early theoretical foundations, subsequent work seeks to formalize and expand the scope of network analysis. Prell (2012) provided a comprehensive account of the historical development, theoretical orientations, and methodological approaches that support network analysis. Their framework positions network theory within broader sociological traditions, including structuralism, role theory, and social capital, while also emphasizing the methodological innovations that make network analysis distinctive. By integrating history, theory, and methodology, Prell (2012) demonstrated how network analysis evolved from

an abstract conceptual perspective into a versatile research program capable of addressing diverse empirical questions across the social sciences.

Beyond its theoretical support, network analysis has inspired a wide range of innovative applications across different disciplines. For example, in computational biology, Horvath and Dong (2008) explored the geometric interpretation of gene co-expression networks, linking network analysis with micro-array data to facilitate the study of gene modules and their interrelationships. Their work demonstrated how network theory can clarify the meaning of, and the connections among, core concepts in gene co-expression analysis. In biology, Wu et al. (2009) introduced the Integrated Network Analysis Platform (PINA), a web-based tool for protein–protein interaction networks. By integrating data from six databases and providing construction, filtering, analysis, and visualization tools, PINA enabled more comprehensive exploration of protein interaction networks. Their results suggested links between LKB1 and TGF β signaling and revealed possible competitive interactors of p53 and c-Jun, emphasising the use of network methods in uncovering biological insights.

Network analysis has also found important applications in the study of international relations. Hafner-Burton et al. (2009) proposed a future programme for applying network analysis to understand international systems. They advocate for adopting network tools to deepen research, test existing theories, and explore new hypotheses, positioning network analysis as a valuable framework for analysing complex international interactions. In another domain, health psychology, Hevey (2018) provided a practical overview tailored to behavioural scientists, illustrating how network analysis can be applied to frameworks such as the Theory of Planned Behaviour. By showing how psychological constructs can be represented and analysed as networks, this work broadened both the accessibility and the methodological reach of network analysis.

Recent developments have extended these applications even further. For instance, Porreca et al. (2024) applied fuzzy centrality measures to academic collaboration networks, demonstrating how fuzzy approaches can capture nuances in centrality that traditional metrics often miss. Likewise, Rimkevičienė et al. (2024) used network analysis to study suicidality, revealing the complex interplay of risk factors within a community sample. Their findings suggested that certain factors, such as acquired capability for suicide, may have weak or no direct links within the network, highlighting the unique insights network analysis can provide in mental health research.

Taken together, these developments emphasize the versatility of network analysis as both a theoretical and applied framework. Across fields as diverse as biology, international relations, psychology, and public health, network analysis enables researchers to uncover structural patterns, dynamic interactions, and hidden mechanisms within complex systems. Its broad relevance and methodological adaptability make it a powerful tool for advancing knowledge across disciplines.

2.3 Network topologies

Network analysis can be applied to a variety of network types. In this study, we focus on three specific topologies: random networks (Erdős–Rényi), small-world networks (Watts–Strogatz) and scale-free networks (Barabási–Albert). The following section reviews existing literature on these network topologies.

2.3.1 Random networks (Erdős–Rényi)

The study of random network topology originates from the pioneering work of Erdős and Rényi, who formally introduced the concept of random graphs in their ground-breaking papers Erdős and Rényi (1959) and Erdős and Rényi (1960). Their model, now widely known as the Erdős–Rényi model, defines networks as sets of nodes in which each potential edge occurs independently with a fixed probability, laying the mathematical foundations for the modern study of network structure and dynamics. Building upon the foundational ideas, Greaves and Davis (1974) developed a continuous random network (CRN) model characterized by three-fold coordination, aiming to better represent the local bonding environment in amorphous solids. This model sought to reconcile the disordered nature of amorphous materials with the constraints of chemical bonding, providing a more detailed structural depiction.

Subsequent developments, most notably the comprehensive treatment by Bollobás (2001) in *Random Graphs*, organised and extended this early research, providing precise probabilistic methods and theoretical results.

In wireless networks, the properties of random topologies influence broadcast efficiency and scalability (Farago, 2009). They investigate the scalability of node degrees within random wireless network topologies, establishing a general modeling framework that encompasses multiple random graph models. The study demonstrates that certain conditions inherently lead to a lack of degree scalability, emphasizing the limitations and challenges associated with large, random wireless networks. Similarly, Faragó (2012) extends this analysis to large-scale wireless networks, Web, and Internet models, focusing on the complexity and vastness of such topologies, which are often characterized by randomness and heterogeneity.

2.3.2 Small-world networks (Watts–Strogatz)

Small-world networks were introduced by Watts and Strogatz (1998), who observed that many real-world networks combine high clustering, typical of regular lattices, with short average path lengths, characteristic of random graphs. Their model demonstrated that even a small fraction of randomly rewired edges could drastically reduce the distance between nodes while preserving local connectivity, capturing a phenomenon widely seen in social, biological, and technological systems. Watts (1999) later expanded on this framework, providing both theoretical insights and empirical examples of networks that exhibit small-world properties. This work highlighted the balance between order and randomness in network topology and has since influenced research across multiple disciplines.

The concept of small-world network topology has accumulated significant attention across various fields, highlighting its unique structural properties and functional implications. Christley and French (2003) emphasized the importance of network topology in understanding infectious disease dynamics, illustrating how the small-world structure of contact networks in the UK racing industry facilitates rapid spread of infectious agents. This highlights the relevance of small-world networks in modeling real-world contact patterns that influence disease transmission.

In computational contexts, Tomassini et al. (2004) shows that small-world network topologies consistently evolve from regular and random structures without being designed beforehand by using an evolutionary algorithm, particularly in automata systems. These topologies have better performance than regular lattice structures and are easier to evolve, which later explains in part their ubiquity (Tomassini et al., 2004). Theoretical explorations extend into high-energy physics, where El Naschie (2004) discussed the relationship between small-world networks, the $E(\infty)$ topology, and the mass spectrum of high-energy particles. This indicates the broad applicability of small-world concepts beyond biological and computational systems.

2.3.3 Scale-free networks (Barabási–Albert)

Scale-free networks were first identified by Barabási and Albert (1999), who observed that many real-world networks – such as the Internet, social networks, and biological systems – exhibit a power-law degree distribution. In these networks, a few highly connected nodes, or “hubs”, dominate connectivity, while most nodes have few links, a pattern not captured by classical random networks (Erdős–Rényi). The mechanism underlying this structure was formalized as preferential attachment, in which new nodes are more likely to connect to already well-connected nodes (Barabási and Albert, 1999). Subsequent work by Albert and Barabási (2002) provided a comprehensive review of the statistical mechanics of complex networks, consolidating the theoretical foundations and applications of scale-free networks across disciplines.

The concept of scale-free network topology is extensively explored across various domains, highlighting its fundamental structural properties and practical implications. Smith (2002) demonstrates that instant messaging networks inherently exhibit scale-free characteristics, suggesting that such topologies are prevalent in digital communication systems. Similarly, Wu et al. (2004) identifies scale-free features within urban transit systems, specifically analysing the Beijing transit network. Their findings indicate that controlling connected nodes within these networks enhances resilience against random failures, such as traffic congestion or accidents, emphasising the robustness conferred by scale-free structures. Further theoretical insights are provided by Papadopoulos et al. (2010), who reveals that complex scale-free networks naturally emerge from hyperbolic metric spaces and shows that greedy forwarding algorithms can reliably find shortest paths with high probability within such topologies. The topological structure of a network has a marked influence on the processes that may occur

within it. The next section explores how these processes operate within the specific network structures analysed in this study.

2.4 Community detection

A central challenge within network analysis is the detection and analysis of community structures – clusters of nodes with dense internal connections relative to their links with other parts of the network. Community detection has garnered significant attention across various fields, reflecting its importance in understanding complex systems. Several algorithms have therefore been proposed to find reasonably good partitions efficiently.

Figure 2.1 illustrates a generated random network with ten nodes and twenty edges. The left panel shows the network before community detection, while the right panel presents the network after community detection has been performed. The different colours represent distinct communities, highlighting how the algorithm partitions the network into groups of densely connected nodes.

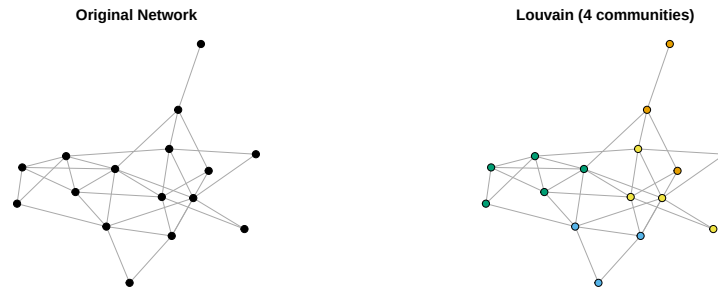


FIGURE 2.1: A generated random network before (left) and after (right) community detection. Distinct colours represent the communities identified by the detection algorithm, revealing groups of densely connected nodes.

Community detection in large networks is a time-consuming and computationally complex task due to the size variations and the unavailability of the number of communities (Fortunato and Darko, 2016). Several types of community detection algorithms exist. Divisive algorithms identify and remove inter-community links, agglomerative algorithms recursively merge similar nodes or communities, and optimization methods aim to maximize a predefined objective function, often directly related to partition quality (Blondel et al., 2008). To evaluate the performance of these algorithms, quality functions are used. A widely adopted measure is *modularity*, a scalar value between -1 and 1 that quantifies the density of links within communities relative to links between communities (Blondel et al., 2008). Optimization-based algorithms frequently aim to maximize modularity, which serves both as a guiding objective and a key metric for assessing the quality of detected communities, as discussed further in Section 3.3.1.

Khatoon and Banu (2021) described the core objective of community detection as partitioning networks into dense regions corresponding to entities with close relationships, thus forming

communities. In essence, the goal of community detection is to partition the network into dense regions, where those regions correspond to closely related entities that can be said to belong to the same community (Khatoon and Banu, 2021).

A comprehensive literature survey has been presented for social network analysis and community detection, highlighting the limitations prevalent in current technologies. In 2013, Deitrick et al. (2013) emphasised the relevance of community detection in social media, particularly Twitter, where it enhances sentiment analysis by identifying groups of interacting users. Their study adopted an integrative approach, using community detection output to enable community-level sentiment analysis. Community detection was performed using the Walktrap algorithm on a network of Twitter users associated with Microsoft Corporation's @technet account, which is primarily used for communication purposes (Deitrick et al., 2013).

Ostovari and Yu (2019) investigated the impact of care provider network characteristics on patient outcomes through the use of social network analysis and a multi-scale community detection algorithm. Mucha et al. (2010) developed a generalized framework for community detection in complex, multi-layered networks, known as multi-slice networks. Their approach allows for the study of community structures in networks that are composed of interconnected slices, each representing different types or states of the network, thus accommodating the dynamic and multi-scale nature of real-world systems.

Zhu et al. (2023) proposed a multi-modal, non-Euclidean brain network analysis framework that combines community detection with convolutional auto-encoders. Jin et al. (2023) addressed the growing complexity of network analysis by proposing a unified framework for community detection methods. They introduced a new taxonomy categorizing existing approaches into probabilistic graphical models and deep learning-based techniques. The study provided a comprehensive review of current methods, released benchmark datasets for evaluation, and discussed key theoretical and methodological challenges that remain critical for advancing the field.

Christensen et al. (2024) conducted a Monte Carlo simulation to compare different community detection algorithms within psychological networks. Their study performed a Monte Carlo simulation using the zero-order correlation matrix, GLASSO, and two variants of non-regularized partial correlation sparsity induction methods with several community detection algorithms, and their findings indicate that the Fast-Greedy, Louvain, and Walktrap algorithms, when combined with the GLASSO method, were consistently among the most accurate and least biased across both continuous and polytomous data conditions.

Overall, these studies collectively demonstrate the broad applicability of community detection across diverse domains, from social media and healthcare to neuroscience and psychology. Attention now shifts to the literature on the two community detection algorithms central to this study, namely the Louvain and Leiden algorithms.

2.4.1 The Louvain algorithm

The Louvain algorithm, developed by Blondel et al. (2008), is a multi-phase, iterative heuristic for optimising modularity that has become one of the most widely used algorithms for community detection. The algorithm finds high modularity partitions of large networks in a short time and unfolds a complete hierarchical community structure for the network, thereby giving access to different resolutions of community detection. It efficiently uncovers high-modularity partitions in large networks while maintaining low computational and memory costs (Blondel et al., 2008).

The algorithm operates in two main phases, namely, the local moving of nodes and aggregation of the network. These two phases are repeated iteratively. Initially, each node is assigned to its own community. For each node i , the algorithm evaluates the modularity gain resulting from moving i into the community of each of its neighbours j . The node is then reassigned to the community that yields the highest positive gain in modularity. This process continues sequentially for all nodes until no further improvement is possible, completing the first phase. In the second phase, communities are aggregated into meta-nodes, forming a reduced network on which the process is repeated. These phases continue iteratively until modularity converges, yielding a hierarchical community structure (Blondel et al., 2008). The Louvain method is highly scalable, with computational limits determined primarily by storage capacity rather than processing time (Blondel et al., 2008).

Figure 2.2 provides a graphical representation of how the Louvain algorithm optimizes a quality function, such as modularity or the Constant Potts Model (CPM), in the two fundamental phases: (1) local moving of nodes, and (2) aggregation of the network. In the local moving phase, individual nodes are moved to the community that yields the largest increase in the quality function. In the aggregation phase, an aggregate network is created based on the partition obtained in the local moving phase, where each community becomes a node in the reduced network. These phases are repeated until the quality function can no longer be increased.

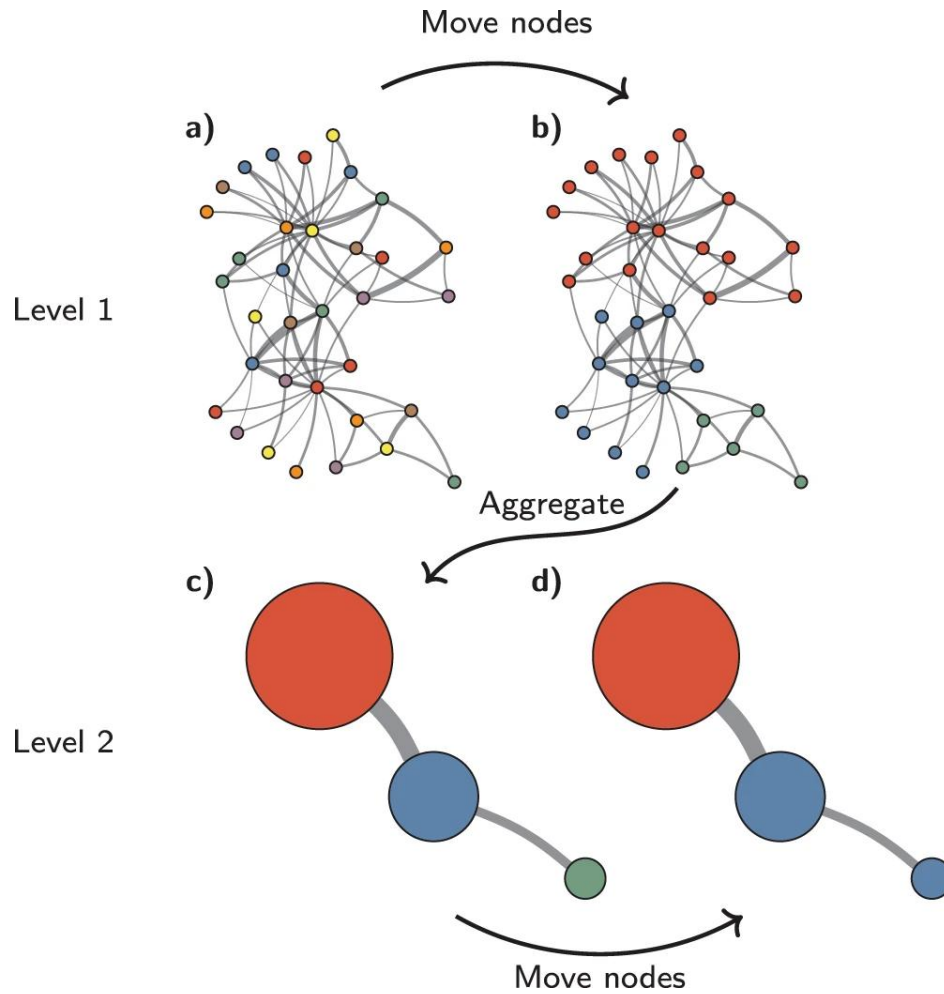


FIGURE 2.2: Schematic illustration of the Louvain algorithm's two main phases: (1) local moving of nodes, and (2) aggregation of the network. The Louvain algorithm starts from a singleton partition in which each node is in its own community (a). The algorithm moves individual nodes from one community to another to find a partition (b). Based on this partition, an aggregate network is created (c). The algorithm then moves individual nodes in the aggregate network (d). These steps are repeated until the quality cannot be increased further (Traag et al., 2019).

One of the key limitations of the Louvain algorithm is its tendency to form badly connected communities. Specifically, the Louvain algorithm may identify communities that are internally disconnected, meaning that one part of such a community can only be reached from another part only through a path that goes outside the community itself. Most importantly, Traag et al. (2019) demonstrated that the problem of disconnected communities is not just a theoretical curiosity through experimental analysis. The problem occurs frequently in practice when using the Louvain algorithm. Perhaps surprisingly, iterating the algorithm aggravates the problem, even though it does increase the quality function (Traag et al., 2019). Figure 2.3 below shows an example a network where the Louvain detects a disconnected community.

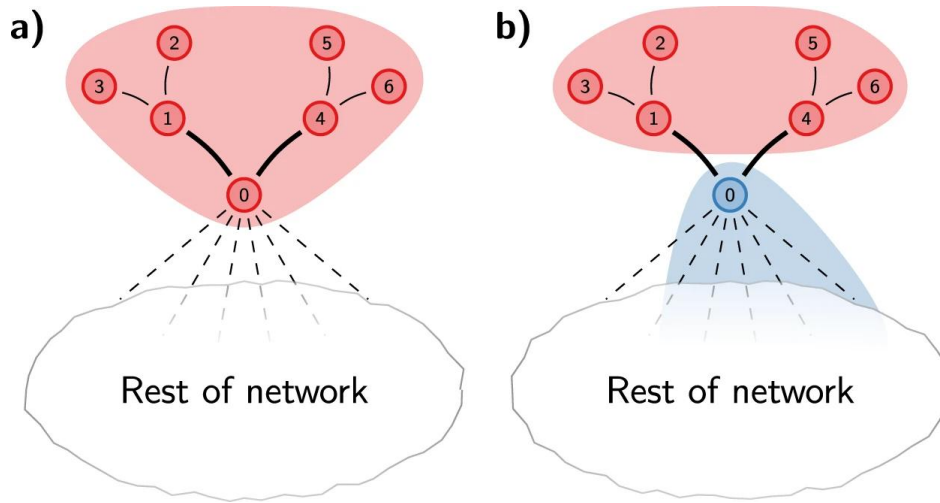


FIGURE 2.3: The thickness of the edges represents the weight. The number in the node is their ID. The background shading represents the community nodes belong to Disconnected community. Consider the partition shown in (a). When node 0 is moved to a different community, the red community becomes internally disconnected, as shown in (b). However, nodes 1–6 are still locally optimally assigned, and therefore these nodes will stay in the red community. (Traag et al., 2019).

While the standard Louvain algorithm is simple and effective (Traag et al., 2019), several enhancements have been proposed to improve scalability and efficiency. Lu et al. (2015) introduced heuristics for parallelizing the Louvain method, achieving higher modularity for many inputs while allowing linear scaling up to 16 threads and real speed-ups using up to 32 threads. Beyond CPU parallelization, additional computational improvements have been made. Mohammadi et al. (2021) proposed an adaptive CUDA-based Louvain algorithm leveraging GPU acceleration, significantly reducing computation time for large networks. Hong et al. (2021) integrated Louvain into a multi-level graph drawing framework, facilitating the visualization of large graphs by detecting communities at multiple hierarchical levels. More recently, Wierzbinski et al. (2023) explored the application of quantum computing to enhance community detection using the Louvain algorithm. One notable variant of the Louvain algorithm – the focus of this study – is the Leiden algorithm.

2.4.2 The Leiden Algorithm

The Leiden algorithm, introduced by Traag et al. (2019), was developed as an improvement over the Louvain algorithm, addressing one of Louvain’s key limitations – its tendency to produce badly connected communities, characterized by few internal edges and many connections to the rest of the network (other communities). The Leiden algorithm guarantees that communities are well-connected, providing more stable and faster community detection results. It is partly based on the previously proposed Smart Local Moving algorithm (Waltman and Eck, 2013), which itself was an enhancement of the Louvain approach. The Leiden method incorporates several important innovations, including the acceleration of local node

movements (Bae et al., 2017; Ozaki et al., 2016) and the introduction of random neighbour selection (Traag, 2015), both of which represent promising directions for improving Louvain’s performance.

According to Traag et al. (2019), the Leiden algorithm consists of three main phases: (1) *local moving of nodes*, (2) *refinement of the partition*, and (3) *aggregation of the network* based on the refined partition. In contrast to Louvain, which immediately aggregates communities after the local moving phase, Leiden introduces a refinement phase that ensures each community is internally well-connected before aggregation. This refinement step enhances the quality of the detected communities and ensures that the resulting partitions meet specific structural guarantees.

In the refinement phase, an initial singleton partition is created in which each node forms its own community. Nodes are then locally merged with other communities, but only within the boundaries of their parent communities from the previous partition. Mergers occur only when both the node and the target community are sufficiently well-connected to their respective parent communities. Unlike Louvain, which merges nodes greedily based on the largest modularity gain, Leiden employs a probabilistic approach. A node may be merged with any neighbouring community that increases the quality function, with the probability of selection depending on the magnitude of the gain. This stochasticity, controlled by a parameter $\theta > 0$, allows the algorithm to explore the partition space more broadly while still avoiding decreases in the quality function, leading to more robust community detection (Traag et al., 2019).

As explained by Traag et al. (2019), another key distinction lies in the implementation of the local moving phase. Whereas Louvain repeatedly visits all nodes in the network until no further improvement is possible, Leiden employs a fast local move procedure that only revisits nodes whose neighbourhoods have changed. This queue-based updating mechanism, similar to “pruning” and “prioritisation” heuristics, significantly reduces computational redundancy. In this procedure, all nodes are initially placed in a queue in random order. Nodes are then sequentially removed, and their movement to a different community is considered only if it increases the quality function. When a node is moved, its affected neighbours are added back into the queue. This continues until the queue is empty, resulting in a more efficient convergence process compared to Louvain (Traag et al., 2019).

Finally, the Leiden algorithm provides several important guarantees. It is executed iteratively, using the partition from one iteration as the starting point for the next. At various stages, the algorithm ensures that all identified communities are well connected internally and locally optimized with respect to the quality function. These properties make the Leiden algorithm a robust and theoretically sound improvement over Louvain, combining efficiency, stability, and quality of results (Traag et al., 2019). Figure 2.4 shows a schematic illustration of the Leiden algorithms:

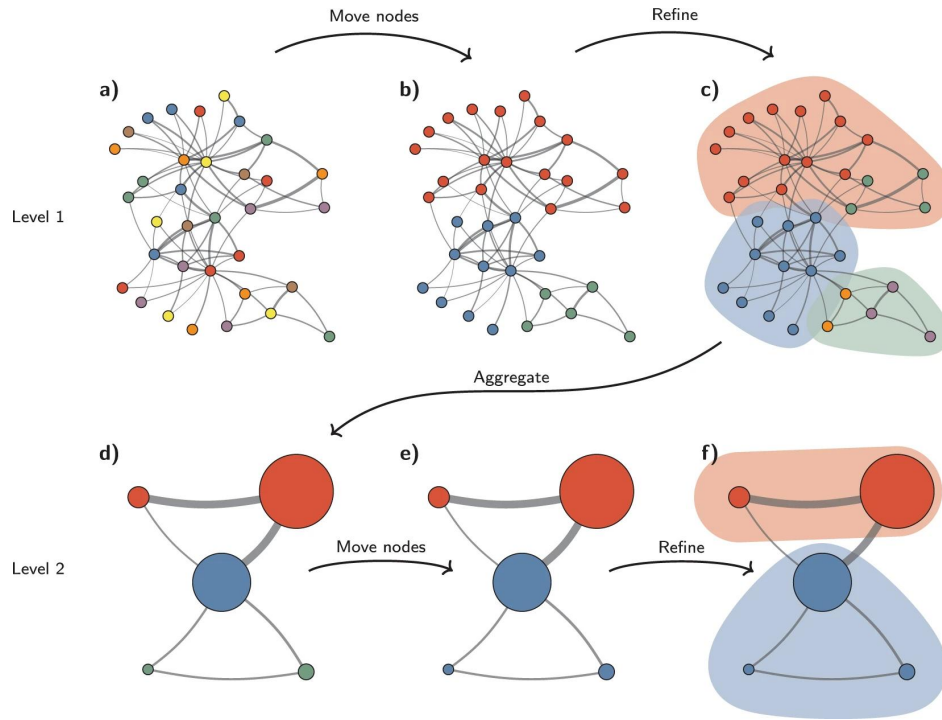


FIGURE 2.4: Leiden algorithm. The Leiden algorithm starts from a singleton partition (a). The algorithm moves individual nodes from one community to another to find a partition (b), which is then refined (c). An aggregate network (d) is created based on the refined partition, using the non-refined partition to create an initial partition for the aggregate network. For example, the red community in (b) is refined into two sub-communities in (c), which, after aggregation, become two separate nodes in (d), both belonging to the same community. The algorithm then moves individual nodes in the aggregate network (e). In this case, refinement does not change the partition (f). These steps are repeated until no further improvements can be made. (Traag et al., 2019)

Hairol Anuar et al. (2021) also provided a comparative review of the Leiden algorithm alongside the Louvain method, emphasizing its foundation in agglomerative techniques and its effectiveness in understanding complex network structures. This review emphasises the algorithm's role in revealing community structures within networks. Empirical studies focusing on modularity metrics also support the Leiden algorithm's efficacy. Hairol Anuar et al. (2024) conducted an empirical comparison between Louvain and Leiden algorithms, emphasizing the latter's advantages in optimizing modularity and detecting well-defined communities. This reinforces the algorithm's reputation for producing high-quality community partitions. Having established the theoretical and empirical context for this study, the next section describes the methodological approach employed to address the gaps identified in the literature.

Chapter 3

Methodology

This study compares the performance of the Louvain and Leiden community detection algorithms across different network topologies. We focus on evaluating quality functions such as modularity, conductance, and stability by systematically applying the algorithms to simulated networks of varying sizes, mean degrees, and resolution parameters. In this chapter, we discuss the experimental design, explain how the networks were simulated, and describe the methods used in the analysis. The methodology consists of four main components: network generation, community detection, performance evaluation, and reproducibility and experimental design.

3.1 Network topologies

The networks are generated using R (R Core Team, 2025), with the `igraph` (Csardi and Nepusz, 2006) and `ggraph` (Pedersen, 2024) packages. The `igraph` package creates networks with predefined structures and specified networks, as well as measuring performance using metrics such as modularity. The `ggraph` package is used for the visualisation of the generated networks.

Each generated network is characterized by its mean degree, $\langle k \rangle$, which represents the average number of edges connected to a node. Mean degree is an important factor influencing network density, defined as the proportion of actual edges relative to the total possible edges. As the mean degree increases, more edges are present in the network, resulting in higher density. Consequently, networks with larger mean degrees are denser.

There are three types of network topologies considered, each generated through specific functions in `igraph`:

In generating the various network topologies for this project, both `directed` and `loops` are fixed to `FALSE` in the random and scale-free network topologies. In the small-world network generation, only `loops` are fixed to `FALSE`.

- **Random networks (Erdős–Rényi):** In this project, random networks are generated using the function `sample_gnp`, which is found in the `igraph` package in R. This function

constructs a graph by uniformly sampling from the set of all graphs with a given number of nodes and edges. The key arguments are:

- `n`: number of vertices in the network.
- `p`: The probability for drawing an edge between two arbitrary vertices ($G(n,p)$ graph).

Figure 3.1 shows an example of a random network generated using `sample_gnp`.

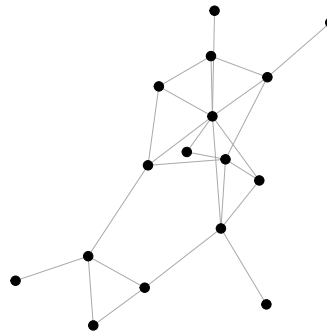


FIGURE 3.1: Example of a random network produced with 16 nodes, a resolution of 1 and a mean degree of 4.

- **Small-world networks (Watts–Strogatz):** The small-world networks in this project were generated using the function `sample_smallworld`. This function captures the small-world properties, characterized by high clustering and short average path lengths. The key arguments are:
 - `size`: number of nodes in the network.
 - `nei`: the neighbourhood within which the vertices of the lattice will be connected.
 - `p`: rewiring probability, it is the probability that we will rewire one end of a given edge to a different, randomly chosen node. This value lies between zero and one.

Figure 3.2 shows an example of a small-world network using the `sample_smallworld` function.

- **Scale-free networks (Barabási–Albert):** The scale-free networks in this project are generated using the function `sample_pa` from the `igraph` package. This model grows networks via preferential attachment, producing degree distributions that follow a power law. The key arguments are:
 - `n`: number of vertices in the network.
 - `m`: number of edges each new node brings when added to the network, thereby influencing the growth rate of hubs.

Figure 3.3 shows an example of a Scale-free network generated using the `sample_pa` function.

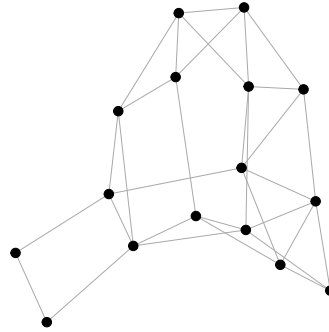


FIGURE 3.2: Small-world network produced with 16 nodes, a mean degree of 4, a resolution of 1 and a rewiring probability of 0.10.

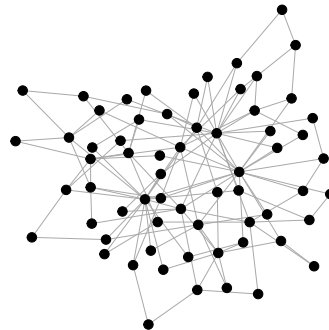


FIGURE 3.3: Scale-free network produced with 64 nodes, a resolution of 1 and a mean degree of 4 specified.

3.2 Community Detection

Community detection is performed using two widely applied algorithms, Louvain and Leiden, as described in Section 2.4, both implemented in R via the `igraph` package. To ensure consistency across analyses, custom wrapper functions were created around the native `cluster_louvain` and `cluster_leiden` functions. These wrappers allowed for systematic adjustment of the resolution parameter (see Section 3.3.1) and ensured that the Leiden algorithm was explicitly configured to optimize modularity. Running both algorithms on the same set of generated networks enabled a direct comparison of their performance in terms of the number of communities detected, modularity, conductance, and stability across varying network topologies and parameters. Each algorithm was executed 100 times per network, and stability was estimated over 50 repeated comparisons per run. The repeated stability assessments were necessary as both Louvain and Leiden are stochastic algorithms, meaning that multiple executions can yield different community assignments.

Figures 3.4 – 3.6 show examples of community detection performed on different network topologies:

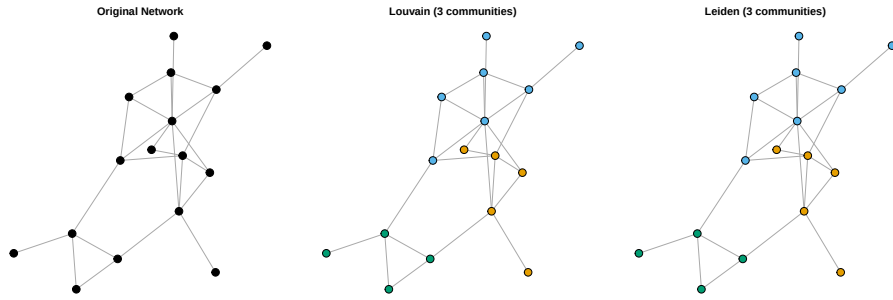


FIGURE 3.4: A random network before community detection and after Louvain and Leiden algorithms are run on the networks.

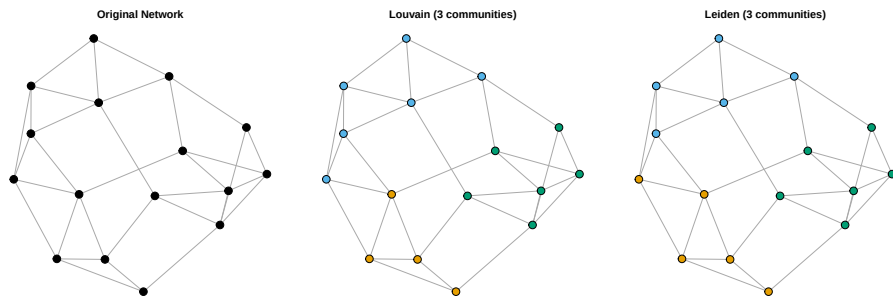


FIGURE 3.5: A small-world network before community detection and after Louvain and Leiden algorithms are run on the network.

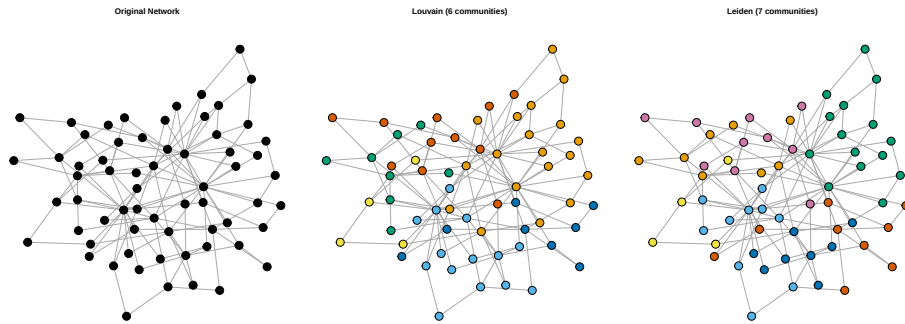


FIGURE 3.6: A scale-free network before community detection and after Louvain and Leiden algorithms are run on the network.

3.3 Assessing community detection algorithms

To measure the performance of the community detection algorithms, we make use of three metrics: modularity, conductance and stability. These metrics capture different aspects of community structure quality: the internal connectivity within communities, their separation from the rest of the network, and the persistence of community assignment across multiple community detection runs on the same network.

3.3.1 Modularity

Modularity is defined as the quality of a particular division of a network (Newman and Girvan, 2004). This metric tries to maximize the difference between the actual number of edges in the community and the expected number of such edges (Traag et al., 2019). According to Reichardt and Bornholdt (2006), modularity can be expressed as:

$$\mathcal{H} = \frac{1}{2m} \sum_c \left[E_c - \gamma \frac{k_c^2}{2m} \right]$$

where

- m : Total number of edges in the network.
- E_c : Actual number of edges within community c .
- k_c : Sum of the degrees of the nodes in community c .
- γ : Resolution parameter (see 3.3.1) used in the community detection, where it controls the size of detected communities.

The resolution parameter, denoted by γ , plays a crucial role:

- When γ is low, the metric tends to favour larger communities, because merging them increases modularity more.
- When γ is high, modularity becomes more sensitive to smaller communities, potentially splitting the network into many small clusters.

However, modularity optimization is known to suffer from the resolution limit problem (Fortunato and Barthélemy, 2007). This limitation means that modularity may fail to identify small communities even when they are clearly defined. Modularity-based algorithms iteratively move nodes between communities to maximize the total modularity of the network. As small communities contribute only minimally to this value, merging them into larger ones often increases the total modularity if the network is large and γ is not set appropriately, leading to the systematic disappearance of small but well-defined communities.

To calculate modularity in **R**, we can make use of the `modularity` function contained in the **igraph** package. This function requires a network topology of your choice and the membership vector returned by a community detection algorithm. Ideally, we aim to maximize this metric.

3.3.2 Conductance

Conductance assesses how well separated a community is from the rest of the network. Formally, the conductance of a graph is defined by Zhang, 2020 as:

$$\phi(G) = \min_{S \subset V} \phi(S) = \min_{S \subset V} \frac{\text{vol}(E(S))}{\min\{\text{vol}(S), \text{vol}(\bar{S})\}} \quad (3.1)$$

where

- $\text{vol}(E(S))$: Number of edges crossing between a subset S of nodes and the rest of the network $\bar{S}$.
- $\text{vol}(S)$: The volume of S , defined as the sum of the degrees (i.e., the total number of edges connected to nodes in S , including edges that leave the subset).
- $\text{vol}(\bar{S})$: The volume of the complement, defined as the sum of the degrees of all nodes outside subset S , including edges connecting into S .

According to Jeub et al. (2015), smaller values of conductance correspond to better communities, as a good community is characterized by nodes that are more densely connected to each other than to the rest of the network.

In practice, instead of evaluating conductance over all possible subsets of nodes, we computed it only for the communities obtained from the Louvain and Leiden algorithms. Specifically:

1. **Community partitioning:** For each algorithm, nodes were grouped by their detected membership labels.
2. **Community-level conductance:** For each detected community S , we calculated $\phi(S)$ using equation 3.1.
3. **Overall conductance score:** We report the mean conductance across all detected communities as an overall measure of community quality. Lower values reflect more well-separated partitions.

3.3.3 Stability

In community detection, stability refers to how consistently an algorithm partitions a network when it is run multiple times under the same settings. A stable algorithm should produce nearly identical partitions across runs, while low stability indicates sensitivity to randomness.

To measure stability, we used the Normalized Mutual Information (NMI), which measures the mutual dependence between two partitions X and Y (Liu et al., 2020). This is expressed numerically by Liu et al. (2020) as:

$$\text{NMI}(X, Y) = \frac{2 I(X; Y)}{H(X) + H(Y)},$$

where

- $I(X; Y)$: the mutual information between partitions X and Y . Mutual information is a quantity that measures a relationship between two random variables that are sampled simultaneously. In particular, it measures how much information is communicated, on average, in one random variable about another (Learned-Miller, 2013).
- $H(\cdot)$: the Shannon entropy of a partition.

NMI values range from 0 to 1, where 0 indicates independent partitions and 1 indicates identical partitions. In practice, we implemented the following procedure:

1. **Multiple runs.** For each algorithm (Louvain and Leiden), the network was partitioned n times using different random seeds.
2. **Membership normalization.** Community labels were standardized so that partitions could be compared directly.
3. **Pairwise similarity.** The similarity between each pair of runs was computed using the Normalized Mutual Information (NMI) metric, via the `aricode` (Chiquet et al., 2023) package, specifically the `NMI` function.
4. **Aggregate score.** The mean similarity across all run pairs was used as the overall stability measure for that algorithm.

This procedure provides a single stability score for each algorithm, allowing for direct comparison of their robustness across repeated runs.

3.4 Reproducibility and Experimental Design

To ensure reproducibility, a deterministic seeding scheme was employed: the random seed for each run was set equal to the run index (e.g., `set.seed(i)`), ensuring that identical network realizations and algorithmic outputs could be reproduced for any given configuration. All intermediate results were stored in `.rds` format, and the entire experimental pipeline – including network generation, community detection, and metric computation – was fully scripted in R. Given the computational demands of performing repeated community detection across multiple network sizes, mean degree, and resolution values, the experiments were parallelized to reduce runtime. The code for these experiments can be found in [this Github project](#).

As seen in Table 3.1, networks were generated across five different sizes: $n \in \{16, 32, 64, 128, 256, 512\}$; four mean degrees: 4, 6, 8, and 10; and a range of resolution parameters: 0.5 to 5.0 in increments of 0.5. The experiments were organized in two complementary stages of analysis.

In the first stage, we varied network size and mean degree while keeping the resolution parameter fixed (the default value being one). This stage allowed us to examine how network size and mean degree jointly influence community detection outcomes.

In the second stage, the resolution parameter was systematically varied between 0.5 and 5 for each network size, with the mean degree held constant. This allowed us to examine how changes in resolution, in combination with network size, influence network partitioning.

For each configuration, networks were generated 100 times to account for random variation. Mean degree was controlled differently depending on the network model: for random networks, mean degree is directly controlled through the edge probability parameter `p`; for scale-free networks, it was controlled indirectly via the parameter `m` (number of edges per

new node); and for small-world networks, the mean degree is controlled indirectly through the neighbourhood size parameter `nei` (number of neighbours per node).

For each network generated, modularity, conductance, and stability metrics were computed. Stability was estimated from 50 repeated community detection runs to capture variability due to the stochastic nature of the algorithms. This experimental design allows for an in-depth evaluation of the Louvain and Leiden algorithms performance across size, mean degree, and resolution dimensions.

TABLE 3.1: Summary of the experimental design for community detection evaluation.

Factor	Values / Settings
Network sizes	16, 32, 64, 128, 256, 512
Mean degree	4, 6, 8, 10
Resolution parameter	0.5 to 5.0 (increments of 0.5)
Experimental stages	Size–Mean degree stage: Evaluation with resolution fixed at a value of one. Size–Resolution–Mean degree stage: Resolution varied within each size–mean degree configuration.
Number of network replications	100 per configuration
Stability repetitions	50 per network run
Algorithms	Louvain, Leiden
Metrics recorded	Modularity, Conductance, Stability, Number of communities

Chapter 4

Results and Discussion

In this chapter, the results are presented, evaluated, and assessed to compare the Louvain and Leiden community detection algorithms based on each performance metric. The chapter consists of two main sections: first, comparing how variations in network size and mean degree affect the performance metrics, and second, comparing the effects of changing the network size and the resolution parameter while holding mean degree constant.

As mentioned in Section 3.4, four mean degree values were investigated. The large volume of results makes it difficult to display all the information clearly, so in Section 4.2, the results where the mean degree value was held constant are provided in the appendix. The results are organized into subsections according to network topology, and in each plot, a comparison is shown between the Louvain and Leiden algorithms. In Section 4.1, the results are illustrated using box plots, while in Section 4.2, they are presented as line graphs with text indicating the average number of communities identified.

4.1 Varying network size and mean degree

In this section, results are presented for networks where both size and mean degree values are varied, visualized using box plots. The y-axis displays the distribution (across 100 simulated networks) of the average metric values (averaged over 50 community detection repetitions), and the x-axis represents the base-2 logarithm of network size. Separate facets correspond to the different mean degree values analysed.

4.1.1 Random networks

Figures 4.1–4.3 summarize how the average modularity, conductance, and stability change across different network sizes and mean degree in random networks.

Modularity

Figure 4.1 shows the average modularity of random networks when network size and mean degree values are varied.

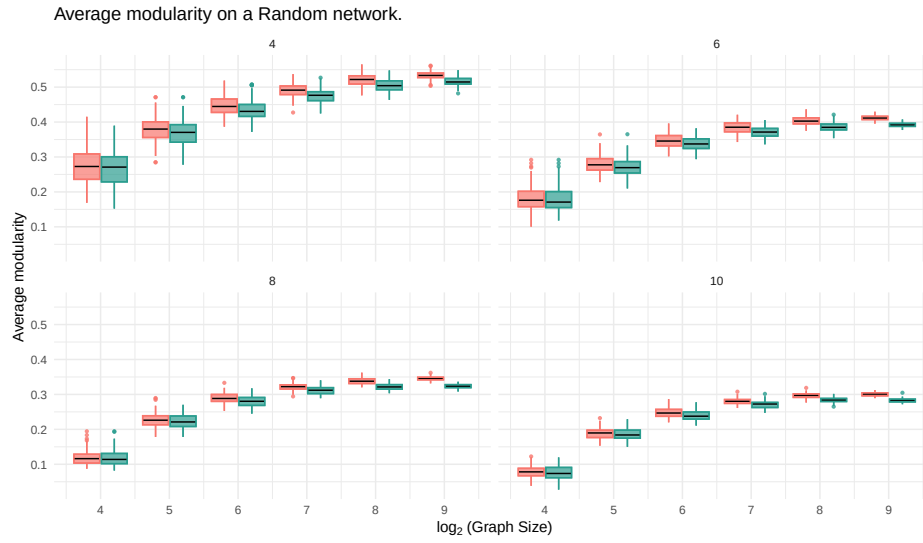


FIGURE 4.1: Average modularity results for random networks of different sizes. The numbers at the top of each box plot represent the mean degree values. Each box plot for a given network size shows the summary of 100 runs. Overall, the Leiden algorithm displays higher average modularity values than the Louvain algorithm.

As shown in Figure 4.1, the modularity values are low, ranging from 0.02 and 0.60. These low values are expected because, in random networks, the number of connections (degree) for each node is similar, and such networks lack meaningful community structure, making it difficult for any algorithm to detect well-separated communities.

The Leiden algorithm consistently produces higher modularity values than the Louvain algorithm across all network sizes. This indicates that the partitions generated by the Leiden algorithm are of higher quality than those produced by the Louvain algorithm. As the mean degree increases, the modularity values decrease due to the greater number of connections between nodes.

Conductance

Figure 4.2 shows the average conductance of random networks when network size and mean degree values are varied. We observe relatively high values, although at a large range, from approximately 0.2 to 0.7. This occurs because the ratio of edges connecting to nodes outside the community is high compared to the number of edges within the community, which is consistent with expectations for random networks. This occurs because random networks lack real community structure. Edges are placed without preference, so nodes are equally likely to connect outside their group as within it, leading to high conductance values. On average, the Leiden algorithm produces slightly lower conductance values than the Louvain algorithm, indicating better partitions across network sizes and mean degree values. The Leiden algorithm provides a modest reduction in conductance, consistent with the uniform and weakly defined community structure typical of random networks.

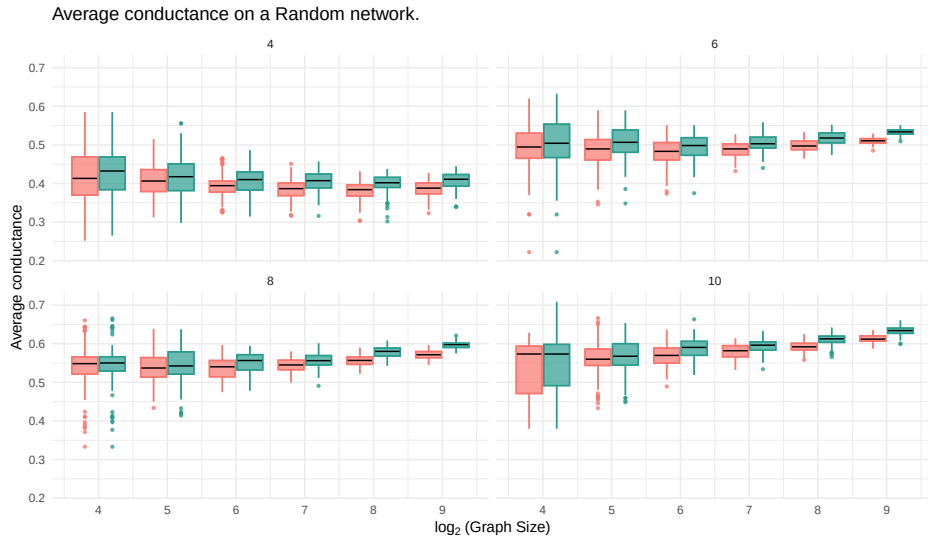


FIGURE 4.2: Average conductance results for random networks of different sizes. The numbers at the top of each box plot represent the mean degree values. Each box plot for a given network size shows the summary of 100 runs. The conductance values are slightly different across network sizes and mean degree values.

Stability

Figure 4.3 shows the average stability of random networks when network size and mean degree values are varied.

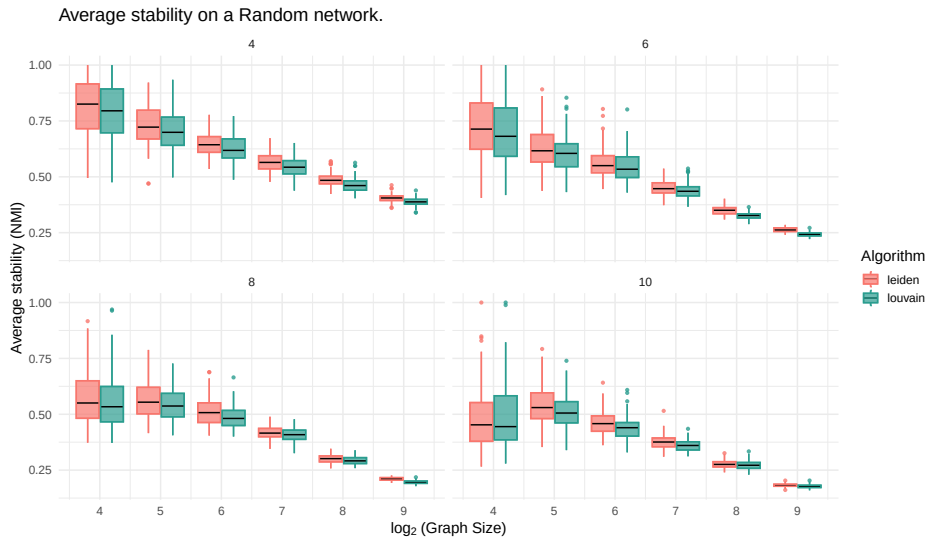


FIGURE 4.3: Average stability results for random networks of different sizes. The numbers at the top of each box plot represent the mean degree values. Each box plot for a given network size summarizes 100 runs, where each run represents the average of 50 stability values. Stability tends to be higher for small networks and lower for larger networks for both algorithms.

In Figure 4.3, the stability values vary widely across network sizes, ranging from 0.16 to

1.00. While the Leiden algorithm produces slightly higher stability values than the Louvain algorithm, small networks with 32 nodes show very high stability, whereas large networks show low stability, especially at higher mean degree values. This indicates that the communities detected in small networks are consistent and reliable across multiple runs, while those in large networks are less stable. This pattern remains consistent as the mean degree varies, and the Leiden algorithm demonstrates a clear stability advantage over the Louvain algorithm in this network topology.

4.1.2 Small-world networks

Figures 4.4–4.6 illustrate the changes in average modularity, conductance, and stability across varying network sizes and mean degree values for small-world networks.

Modularity

Figure 4.4 shows the average modularity of small-world networks when network size and mean degree values are varied.

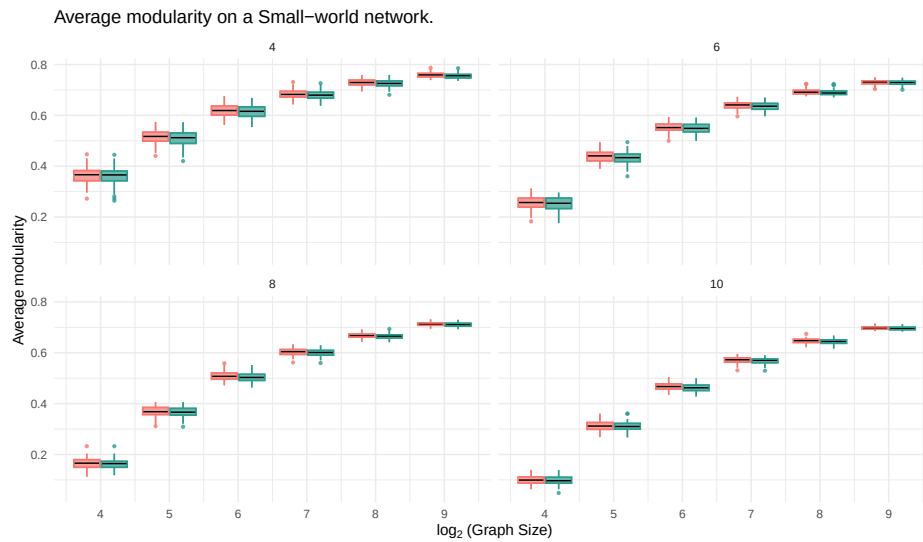


FIGURE 4.4: Average modularity results for small-world networks of different sizes. The numbers at the top of each box plot represent the mean degree values. Each box plot for a given network size shows the summary of 100 runs. The results are slightly different across network sizes and mean degree values. The Leiden algorithm performs slightly better, but the difference in modularity values between the two algorithms becomes very small at larger network sizes.

In Figure 4.4, the modularity values are high, ranging from 0.04 to 0.80. Modularity increases with network size across all mean degree values, indicating that the algorithms effectively detect well-separated communities in small-world networks. This pattern is expected because small-world networks are characterized by high clustering, where nodes form tightly connected groups, and the number of connections between any two nodes remains small despite the network's size. Between the two algorithms, the Leiden algorithm slightly outperforms the Louvain algorithm, achieving higher modularity values across most network sizes. These

results indicate that both algorithms identify strong community structures in small-world networks, with the Leiden algorithm maintaining a slight advantage across network size.

Conductance

Figure 4.5 shows the average conductance of small-world networks when network size and mean degree values are varied.

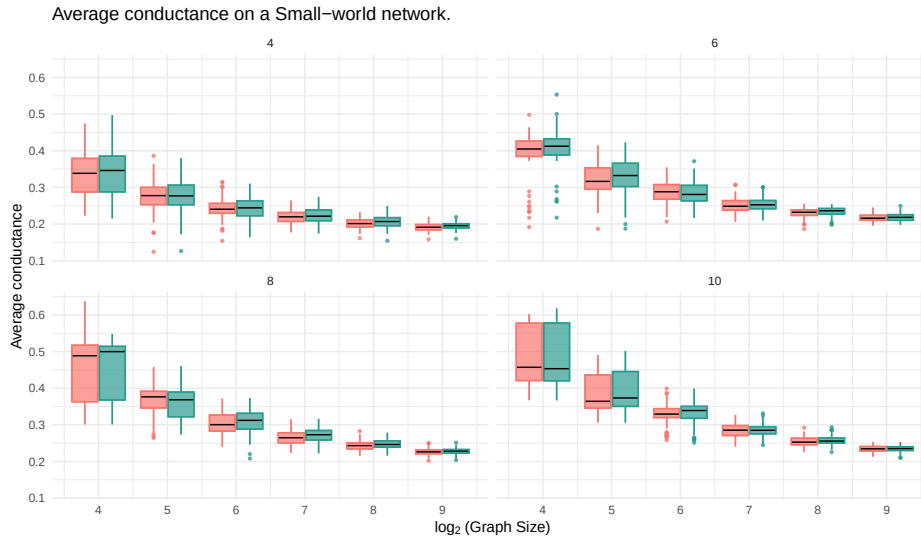


FIGURE 4.5: Average conductance results for small-world networks of different sizes. The numbers at the top of each box plot represent the mean degree values. Each box plot for a given network size shows the summary of 100 runs. The average conductance decreases with increasing network size across all mean degree values.

The conductance values shown in Figure 4.5 are low compared to those of random networks, ranging from 0.12 to 0.60. Contrary to random networks, though, conductance notably decreases as network size increases, with the largest networks (512 nodes) yielding the lowest conductance values across all mean degree values. Therefore, both algorithms detect well-separated communities that are more internally connected and less connected to the rest of the network as the network size increases. The Leiden algorithm again consistently achieves lower conductance values.

Stability

In Figure 4.6, we again observe a wide range of stability values, ranging from approximately 0.29 to 1.00. Interestingly, at an average degree of 4, the stability slightly decreases as network size increases, whereas the stability increases more prominently at higher average degree networks. This is in direct contrast with random networks, where stability decreases for larger networks. The volatile stability at small networks with high average degree might indicate that the simulations of these networks are inconsistent, since a dense, tiny network does not fit the sparsity profile one would expect with small-world networks. Overall, the Leiden algorithm again demonstrates higher average stability than the Louvain algorithm.

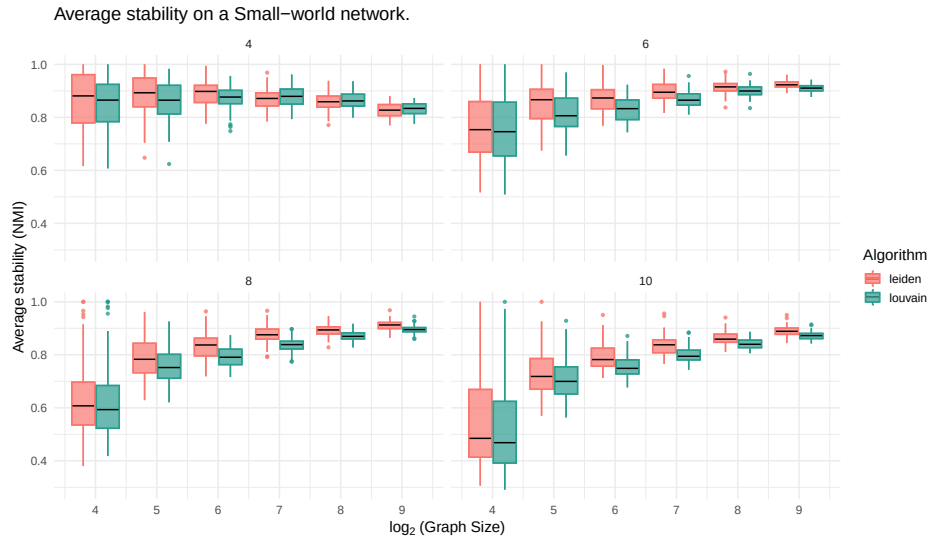


FIGURE 4.6: Average stability results for small-world networks of different sizes. The numbers at the top of each box plot represent the mean degree values. Each box plot for a given network size shows the summary of 100 runs, where each run represents the average of 50 stability values.

4.1.3 Scale-free networks

Figures 4.7–4.9 illustrate the changes in average modularity, conductance, and stability across varying network sizes and mean degree values for scale-free networks.

Modularity

Figure 4.7 shows the average modularity of scale-free networks when network size and mean degree values are varied.

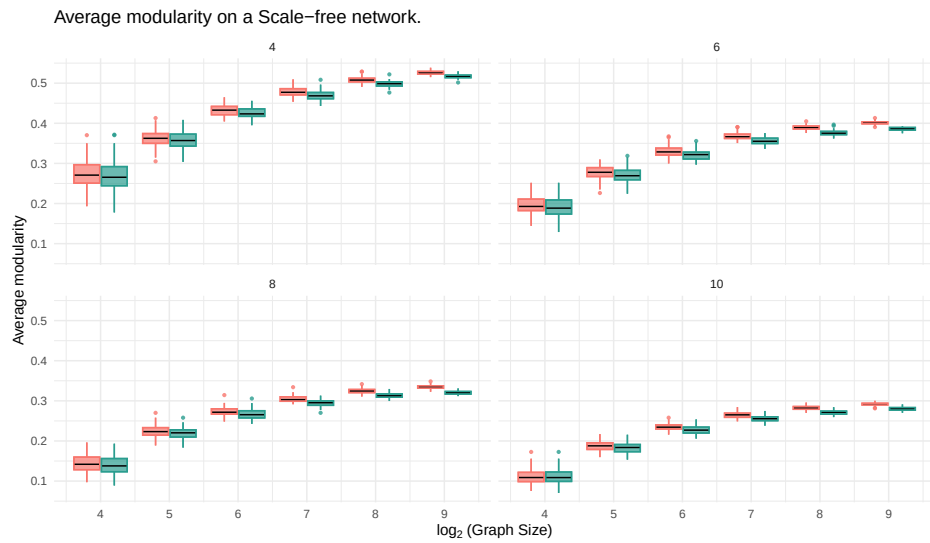


FIGURE 4.7: Average modularity results for scale-free networks of different sizes. The numbers at the top of each box plot represent the mean degree values. Each box plot for a given network size shows the summary of 100 runs. The Leiden algorithm performs slightly better than the Louvain algorithm.

In Figure 4.7, the average modularity values range from 0.07 to 0.53. These values increase with network size across all mean degree values. This pattern is characteristic of scale-free networks, where a few nodes have a very high degree of connectivity (hubs) while most nodes have relatively low degrees. These highly connected hubs make community detection challenging for both algorithms, resulting in lower modularity values. The slight increase in modularity with larger network sizes suggests improved community separation. Between the two algorithms, the Leiden algorithm consistently outperforms the Louvain algorithm across all network sizes and average degrees.

Conductance

Figure 4.8 shows the average conductance of scale-free networks when network size and mean degree values are varied.

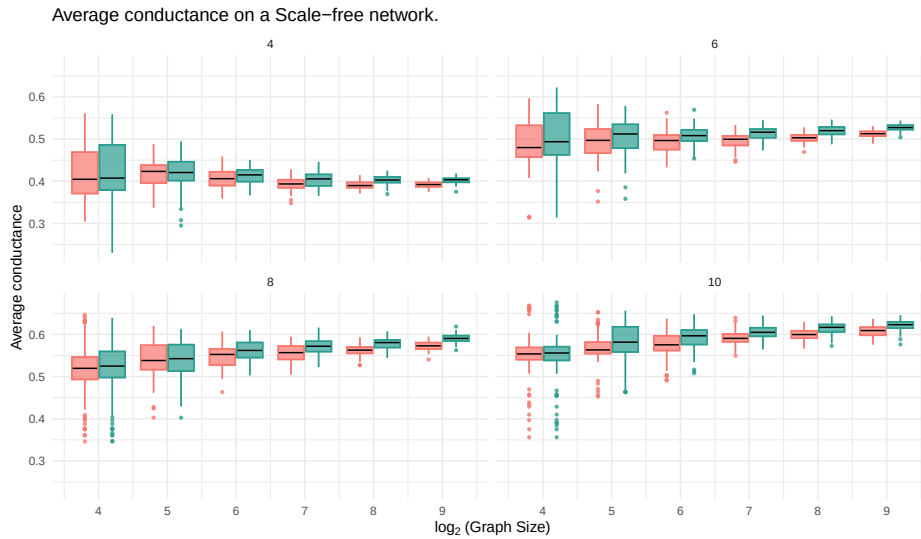


FIGURE 4.8: Average conductance results for scale-free networks of different sizes. The numbers at the top of each box plot represent the mean degree values. Each box plot for a given network size shows the summary of 100 runs.

As shown in Figure 4.8, the average conductance results offer a different view of community separation. The conductance values range from 0.23 to 0.67. The relatively low conductance values in scale-free networks indicate strong connections between communities and weaker separation among them. The Leiden algorithm consistently produces lower conductance values than the Louvain algorithm, suggesting that its detected communities are more internally connected and less connected to the rest of the network. For larger networks, only minor variation in conductance is observed between the two algorithms. As with small-world networks, the large variation in results for tiny networks (16 nodes) with high average degree might be due to inconsistencies in attempting to generate sparse, yet dense networks.

Stability

Figure 4.9 shows the average stability of scale-free networks when network size and mean degree values are varied.

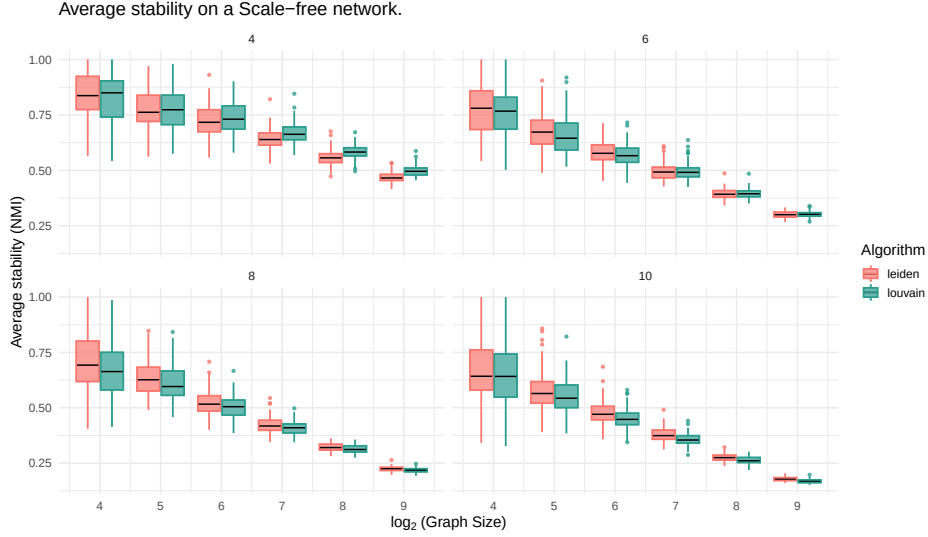


FIGURE 4.9: Average stability results for scale-free networks of different sizes. The numbers at the top of each box plot represent the mean degree values. Each box plot for a given network size shows the summary of 100 runs, where each run represents the average of 50 stability values. The scale-free stability values decrease as the network size increases for all mean degree values.

In Figure 4.9, the average stability values range from 0.20 to 1.00. Overall, the Leiden algorithm achieves higher stability than the Louvain algorithm across all mean degrees, except at a mean degree of four, where Louvain slightly outperforms Leiden. Both algorithms reach the maximum stability value of one at a network size of 16, while the minimum value occurs in the largest network for the Leiden algorithm. These results indicate that the Leiden algorithm generally produces more consistent and reproducible community partitions across runs. As network size increases, stability values tend to decline with higher mean degrees, and differences between the two algorithms become negligible.

Summary of results

Overall, the comparative analysis reveals only minor performance differences between the Leiden and Louvain algorithms across the three network topologies. The Leiden algorithm generally outperforms Louvain across all quality functions, except in scale-free networks at a mean degree of four, where Louvain achieves slightly higher stability. These differences could be explained by the extra refinement step in the Leiden algorithm, which guarantees well-connected communities, and the distinct structural properties of each network topology. All results presented in Section 4.1 were obtained using a fixed resolution parameter of one. The next section examines how varying the resolution parameter while keeping the mean degree constant affects the performance of both algorithms.

4.2 Varying size and resolution at constant mean degree

In this section, we examine how changes in the resolution parameter influence the quality functions – modularity, conductance, and stability – while keeping mean degree constant. The results are presented as line graphs with resolution values on the x-axis and the average metric value on the y-axis. Each plot is faceted by network size, and the accompanying text indicates the average number of communities detected by the respective algorithms.

4.2.1 Random networks

Figures A.1–A.12 summarize how the average modularity, conductance and stability change across network sizes and resolutions while holding the mean degree fixed.

Modularity

Figures A.1–A.4 show the average modularity of random networks as network size and resolution are varied, while keeping the mean degree fixed. A general pattern emerges where modularity tends to increase between resolutions 0.5 and 1, after which it gradually decreases. For small networks (size 16), there is no noticeable difference in the average modularity obtained by the Leiden and Louvain algorithms across all mean degrees. However, as the network size increases, slight differences begin to appear at lower resolutions, as seen in networks of size 32 and 64. From size 128 onward, the Leiden algorithm consistently achieves higher modularity values than Louvain across all resolution values.

Conductance

Figures A.5–A.8 show the average conductance of random networks when network size and resolution are varied while holding the mean degree fixed. The general trend is that as resolution increases, the average conductance increases. At networks of size 16 and 32, there is no difference in the algorithm’s performance across resolutions. At a network of size 64 at low degrees (4 and 6), there is a difference in the algorithm performance, where the Leiden algorithm performs better (has a lower conductance). Networks of size 128 and larger consistently see the Leiden algorithm performing better.

Stability

Figures A.9–A.12 show the average stability of random networks when network size and resolution are varied while holding the mean degree fixed. The trend seen across all network sizes is that the average stability increases as resolution increases. At networks of size 16 and 32, at all mean degrees, there is no difference in the algorithm’s performance. When the mean degree is four and the network size is 64, the Leiden algorithm exhibits higher stability values until a resolution of three is reached. At all other mean degrees, the networks of size 64 appear to have no difference in stability values. The Leiden algorithm has higher stability values in networks of size 128 and larger; these differences begin to become less noticeable at a mean degree of eight and eventually disappear at a mean degree of ten.

4.2.2 Small-world networks

Figures A.13–A.24 summarize how the average modularity, conductance and stability change across network sizes and resolutions while holding the mean degree fixed.

Modularity

Figures A.13–A.16 show the average modularity of small-world networks when network size and resolution are varied while holding the mean degree fixed. All network sizes display an initial increase in modularity between 0.5 and one, then they proceed to consistently decrease. At a mean degree of 6, 8, 10 and network size of 16, the average modularity decreases steeply until a resolution of 3.5, and the modularity becomes constant at a negative value, meaning there are fewer within-community edges than expected. In networks of size 32, there is a steep decrease in the average modularity over all resolutions. In networks of size 64 and larger, this decrease in modularity becomes gradual. In networks of size 128 and larger after a resolution of one, there is a gradual decrease in average conductance. Overall, there is no difference in the Leiden and Louvain algorithm performance.

Conductance

Figures A.17–A.20 show the average conductance of small-world networks when network size and resolution are varied while holding the mean degree fixed. There is an increase in average conductance as resolution increases at all network sizes. When a network is size 16 and mean degree of four, there is a near-linear increase in conductance. This increase becomes more gradual at a resolution of four. In larger mean degrees, there is also this increase, but then the conductance values become constant at a certain resolution; this resolution value becomes lower as the mean degree increases. At networks of large size (128 and above), there is a gradual increase in conductance, but the values reach at most 0.45, which is more ideal compared to lower network sizes, which reach a value of one or very near to it. There is no notable difference in the performance of the Leiden and Louvain algorithms at the four mean degrees tested.

Stability

Figures A.21–A.24 show the average stability of small-world networks when network size and resolution are varied while holding the mean degree fixed. The general trend seen over network sizes is an increase in stability. Initially, at a mean degree of four and a network of size 16, the Leiden algorithm has a higher stability value, although at a resolution of 1.5, the Louvain algorithm performs slightly better and finally at a resolution of four, the Louvain clearly outperforms the Leiden algorithm. At a mean degree of 6, there is only a slight difference between the algorithms. Once again, at a low resolution (below 1.5), the Leiden algorithm performs better; at resolutions above this, the algorithms do not differ. In higher mean degrees, there is no difference in the stability values across all resolutions. In networks of size 32, at all mean degrees besides four, the Leiden algorithm outperforms the Louvain algorithm, but once a resolution of 2.5 has been reached, there is no difference in the algorithm's performance. At a low mean degree of four the Leiden algorithm initially performs better but once a resolution of 2.5 is reached the Louvain algorithm performs better. At

networks of size 64, the Leiden algorithm consistently performs better until a high resolution of four, where both algorithms perform the same at all mean degree values until eight. At a mean degree of ten at a network of size 63 then Leiden algorithm performs better until a resolution of three after which the algorithms perform the same. A network of size 128, at a mean degree of 4, the Leiden algorithm performs better after a resolution of 1.5. In networks of size 128 and larger, but only at mean degrees of size 6 and above, the Leiden algorithm performs better at all resolution values.

4.2.3 Scale-free networks

Figures A.25–A.36 summarize how the average modularity, conductance and stability change across network sizes and resolutions while holding the mean degree fixed.

Modularity

Figures A.25–A.28 show the average modularity of scale-free networks when network size and resolution are varied while holding the mean degree fixed. Overall, the modularity decreases after a resolution of one is reached. It can be seen that the decrease in modularity can approach negative modularity values when a network is small in size (16). In networks of size 16 and 32 at all mean degrees, there is no difference in the algorithm's performance. In networks of size 128 and larger, a slight difference in performance is observed, with the Leiden algorithm performing slightly better.

Conductance

Figures A.29–A.32 show the average conductance of scale-free networks when network size and resolution are varied while holding the mean degree fixed. The conductance increases as the resolution increases across all network sizes. The networks of size 16 and 32 have no differences in algorithm performance. For networks of size 64, the Leiden algorithm performs slightly better as it produces lower average conductance values. Furthermore, in networks sized 128 and larger, we observe that the Louvain algorithm consistently yields higher values. Since we are trying to minimize the conductance, this means the Leiden algorithm performs better.

Stability

Figures A.33–A.36 show the average stability of scale-free networks when network size and resolution are varied while holding the mean degree fixed. The observed pattern is an increase in stability as the resolution increases. At networks of size 16 and 32, the increase is gradual, and the algorithms do not differ. The network of size 64 and mean degrees four and six at a resolution of one and greater, the Louvain algorithm performs microscopically better than the Leiden algorithm. In networks of size 128 and larger, at mean degrees of four and six, where the resolution is greater than one, the Louvain algorithm performs better than the Leiden algorithm. At a mean degree of eight and network size 128, at resolutions above 3.5, the Louvain algorithm performs better. At a mean degree of six and size 128, the Louvain algorithm performs slightly better at higher resolutions. The network size is 512 and the mean degree of six between the resolutions of 2 and 4; the Louvain algorithm once again

performs better than the Leiden algorithm. At a mean degree of 10, the two algorithms have no difference in performance.

4.3 Discussion

When both network size and mean degree were varied (Section 4.1), the results revealed clear and consistent differences in algorithm performance across network topologies. The Leiden algorithm outperformed the Louvain algorithm across all cases except in scale-free networks with a mean degree of four, where Louvain yields slightly higher stability values, though the magnitude of improvement depended on the type of network.

For random networks, both algorithms produced low modularity and higher conductance values, confirming the absence of well-defined communities in such networks. The low modularity values align with theoretical expectations, as random graphs lack inherent community structure. The Leiden algorithm nevertheless achieved marginally higher modularity and lower conductance than the Louvain algorithm, which can be attributed to the refinement step of the Leiden algorithm that ensures well-connected sub-communities. Stability analysis further revealed that smaller random networks (e.g., 32 nodes) yielded more consistent community partitions across runs, whereas larger networks exhibited lower stability. This pattern indicates that stochastic variability in community assignment increases with network size, making larger random networks inherently unstable for community detection.

In small-world networks, both algorithms achieved significantly higher modularity and lower conductance compared to random networks. This outcome is consistent with the high clustering and short path lengths characteristic of small-world structures, which naturally give rise to strong communities. High clustering occurs because nodes tend to form tightly connected groups within the network. Since any node can be reached in just a few steps, these groups remain tightly connected within but only loosely connected to others, which creates distinct communities. The modularity values increased with network size, suggesting that the community structure becomes more pronounced in larger networks. Similarly, conductance decreased with size, implying that communities became more internally well-connected. Stability remained high across all sizes, indicating that the detected community partitions were robust and reproducible. The Leiden algorithm consistently outperformed Louvain, although the difference diminished as the network size became larger. This convergence suggests that for large small-world networks, both algorithms approximate similar community structures.

In scale-free networks, the performance of both algorithms was intermediate between that observed in random and small-world networks. The modularity values were moderate, reflecting the uneven structure of scale-free networks, where a few highly connected nodes (hubs) dominate. These hubs connect many parts of the network, reducing the separation between communities and resulting in lower modularity and higher conductance. Nevertheless, the Leiden algorithm consistently produced higher modularity and lower conductance values than the Louvain algorithm. Stability decreased with increasing network size, suggesting that the

presence of hubs makes community structure less stable, since highly connected nodes can connect multiple regions of the network, causing small changes to affect detected communities. Finally, these findings confirm that network topology strongly influences community quality metrics.

When the resolution parameter was varied while keeping the mean degree constant (Section 4.2), distinct trends emerged across the three network types. The resolution parameter controls the granularity of detected communities, with lower values favouring larger communities, whereas higher values result in smaller, more fragmented partitions.

In random networks, the patterns of modularity, conductance, and stability follow theoretical expectations for structures with no real community structure. Modularity tends to increase slightly at low resolutions (0.5-1) before gradually decreasing as resolution rises. This occurs because higher resolution values divide the network into smaller, arbitrary groups that lack strong internal connectivity. Conductance increases with resolution, reflecting that smaller, resolution-driven communities have a higher proportion of external edges relative to internal ones. The Leiden algorithm performs slightly better (lower conductance) than Louvain for larger networks, but the difference remains minor. Stability generally increases with resolution, suggesting that finer partitions remain relatively consistent. Leiden tends to produce slightly higher stability values in larger networks. Overall, random networks show that both algorithms perform similarly, with Leiden offering small improvements, possibly due to its refinement step. These findings confirm that random networks do not exhibit clear or strong community structure.

For small-world networks, the results reflect the strong clustering and short path lengths typical of these structures. Modularity initially increases at low resolutions but then decreases as resolution rises, particularly in smaller networks. This behaviour is expected because small-world networks contain tightly interconnected groups that may merge or fragment depending on the resolution parameter value. Conductance increases with resolution across all network sizes, although the values remain moderate in larger networks. This indicates that small-world networks maintain well-defined communities even at higher resolutions. Stability also increases with resolution, showing that the detected communities remain consistent. The Leiden algorithm performs slightly better in most cases, particularly at lower resolutions and smaller mean degrees. However, differences between algorithms become negligible in larger networks or at higher mean degrees.

Scale-free networks display distinct behaviour due to the presence of highly connected nodes (hubs) that dominate their structure. Modularity generally decreases after a resolution of one, and in smaller networks, it can even become negative. This suggests that the strong influence of hubs reduces the separation between communities. Conductance increases with resolution but remains relatively low overall, indicating that communities are more connected to each other and less distinct. This can be expected in hub-dominated networks because highly connected hub nodes connect multiple parts of the network, increasing overlap between groups and making community boundaries less distinct. The Leiden algorithm usually achieves lower

conductance than Louvain, implying slightly better-defined communities. Stability increases with resolution, but Louvain often performs marginally better in larger networks and at higher mean degrees. This may occur because Leiden’s refinement step divides hub-centered structures that Louvain tends to preserve.

Across all network types, three main trends are evident. First, increasing the resolution generally increases conductance and stability while decreasing modularity. Second, differences between algorithms become more noticeable in larger networks, with Leiden typically performing slightly better, most likely due to its refinement step. Third, the underlying network topology plays a key role: small-world networks exhibit the clearest community structure, scale-free networks show moderate structure influenced by hubs, and random networks display little to no community structure. These findings demonstrate that the network’s structure has a strong impact on both community quality metrics and the performance of the Louvain and Leiden algorithms. While Leiden often yields modest improvements in modularity, conductance, and stability, the algorithms perform similarly overall.

Chapter 5

Conclusion

This study evaluated and compared the performance of the Louvain and Leiden community detection algorithms across three network topologies: random, small-world, and scale-free. The study systematically examined how variations in network size, mean degree, and resolution parameters affected the algorithms performance, as measured by modularity, conductance, and stability. The results presented in Chapter 4 show that the Leiden algorithm consistently outperformed the Louvain algorithm in detecting communities across all tested scenarios except in a scale-free network at a mean degree of four, where the stability is higher. This confirms the findings of Traag et al. (2019).

This advantage was evident through higher modularity, lower conductance, and greater stability values in most cases, confirming the Leiden algorithm's ability to produce well-connected and reproducible communities. The key factor behind this improvement could lie in the Leiden algorithm's refinement step, which prevents the formation of internally disconnected communities. Despite this difference, both algorithms reflected similar overall trends across network types, indicating that they are each capable of capturing the underlying structural characteristics of complex networks.

The comparative results across topologies revealed clear distinctions driven by network structure. Random networks, as expected, lacked meaningful community structure, producing low modularity, high conductance, and low stability values. Small-world networks displayed strong and stable community structures due to their high clustering and short path lengths, resulting in high modularity, low conductance and high stability values. Scale-free networks, characterized by highly connected hub nodes, produced moderate modularity, moderate conductance, and moderate stability values.

As network size, mean degree, and resolution varied, overall metric values fluctuated in response to structural changes. Nevertheless, the Leiden algorithm consistently maintained an advantage over the Louvain algorithm except at a mean degree of four of a scale-free network, where the stability is better for the Louvain algorithm. The findings highlight the significant influence of network topology, resolution settings, and network sizes on community detection outcomes.

5.1 Challenges and Limitations

Several challenges arose during this project. A challenge was the computational cost. The Leiden and Louvain algorithms perform iterative updates that increase processing time as network size grows. Running multiple simulations across different topologies and sizes required substantial computational power, although this demand remains moderate compared to methods such as Markov Chain Monte Carlo simulations.

Lastly, this study focused on synthetic networks rather than real-world networks. While this allowed for control over network properties, it limited the generalisability of the findings. Real-world networks often contain noise, missing data, overlapping communities, and non-uniform degree distributions that differ from idealized models.

5.2 Future Work and Recommendations

A list of some suggestions that were found during the course of finishing this study is provided below. While this study provides valuable insights into the comparative performance of the Louvain and Leiden algorithms across different network types, several avenues remain open for further research. Future work could expand upon the current findings by exploring additional parameter configurations, larger network scales, and broader algorithmic comparisons to deepen understanding of community detection behaviour under diverse conditions.

Modifying the Rewiring Probability in Small-World Networks

The small-world networks used in this research were generated with a constant rewiring probability, which determines the extent to which local clustering and global shortcuts coexist. Since this parameter controls the balance between regular lattice structure and randomness, varying it could reveal how different levels of small-world influence community detection. A lower rewiring probability preserves strong local clustering, potentially enhancing modularity, while higher values introduce more randomness and weaken community separability. Future work should therefore explore a spectrum of rewiring probabilities to assess how the Louvain and Leiden algorithms adapt to this spectrum.

Exploring Larger Network Sizes

Although this study analysed networks up to 512 nodes, community detection algorithms are often applied to much larger real-world systems. Extending the analysis to include networks with 1000 nodes or more, particularly for random and small-world topologies, would help evaluate algorithm scalability and performance in extreme conditions. Larger networks could also reveal whether the trends observed here, such as the decline in modularity and the rise in conductance with network size, persist or stabilize at higher scales.

Performing Community Detection on Real-World networks

Future studies should extend this analysis to real-world networks to improve the external validity of the findings. Suitable real-world datasets include social networks, communication

networks, transportation systems, and biological interaction networks. Applying the Leiden and Louvain algorithms to such data would provide a more comprehensive evaluation of their effectiveness.

Expanding to directed and weighted networks

Expanding to directed and weighted networks would improve external validity. Both algorithms can handle weighted edges, but their optimization behaviour might differ from the unweighted case. Evaluating these settings can reveal whether Leiden's refinement advantage persists when edge strength or directionality influences community structure.

Appendix A

Appendix

A.1 Random networks at a fixed mean degree

Modularity

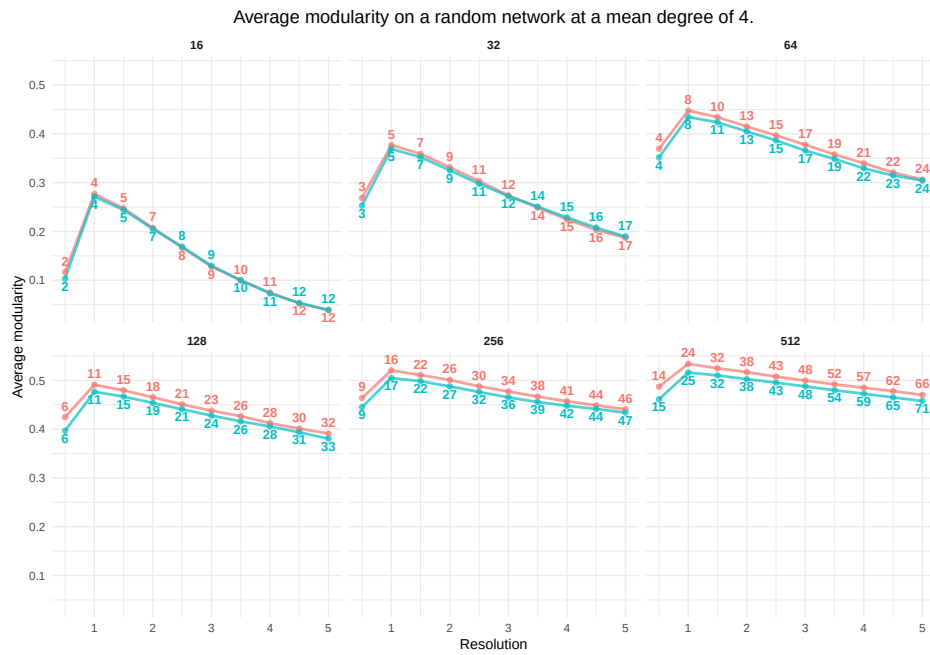


FIGURE A.1: Average modularity results for random networks at a mean degree of four. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

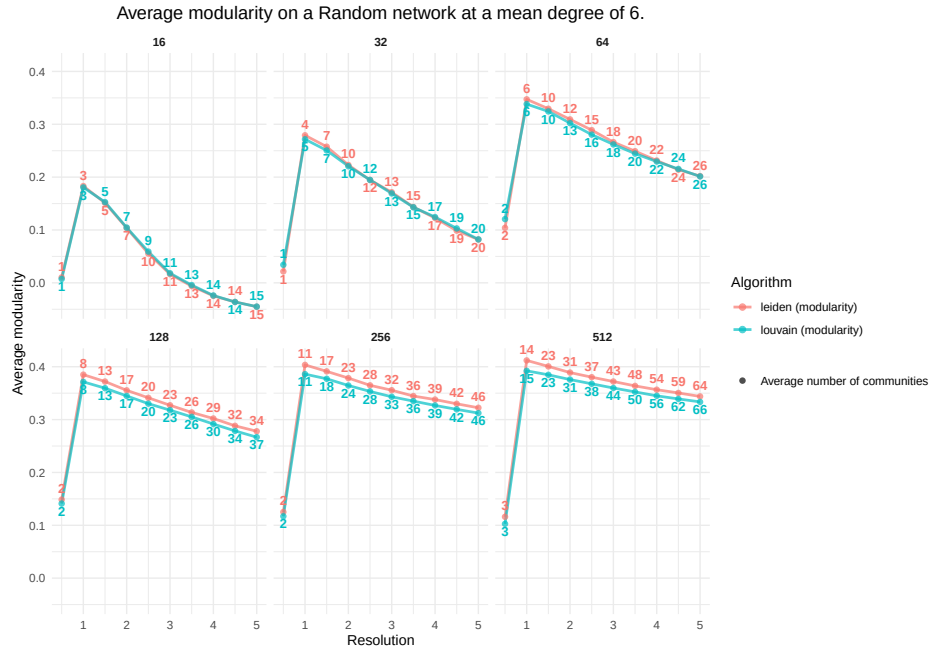


FIGURE A.2: Average modularity results for random networks at a mean degree of six. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

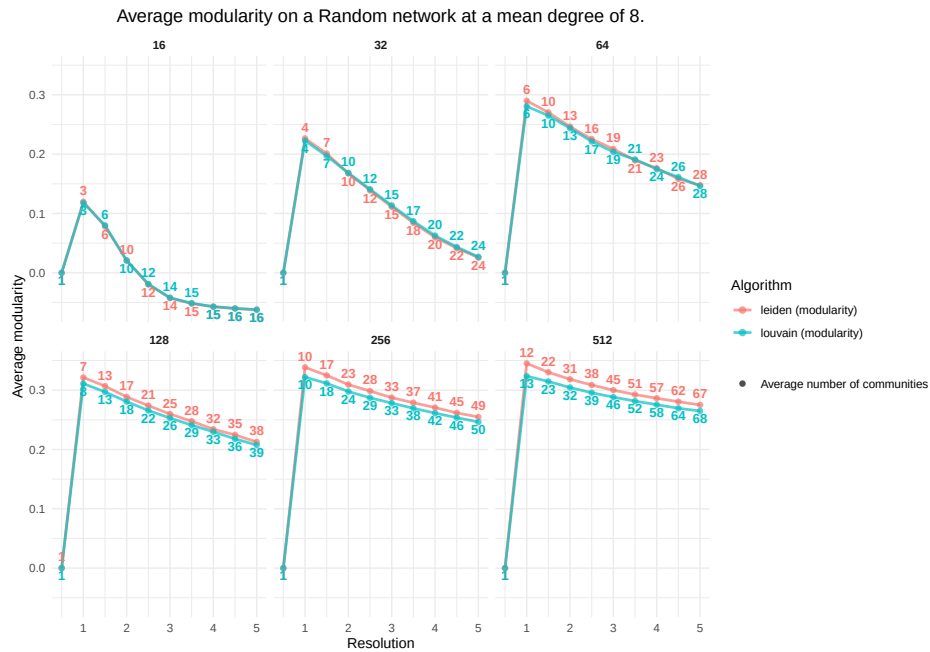


FIGURE A.3: Average modularity results for random networks at a mean degree of eight. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

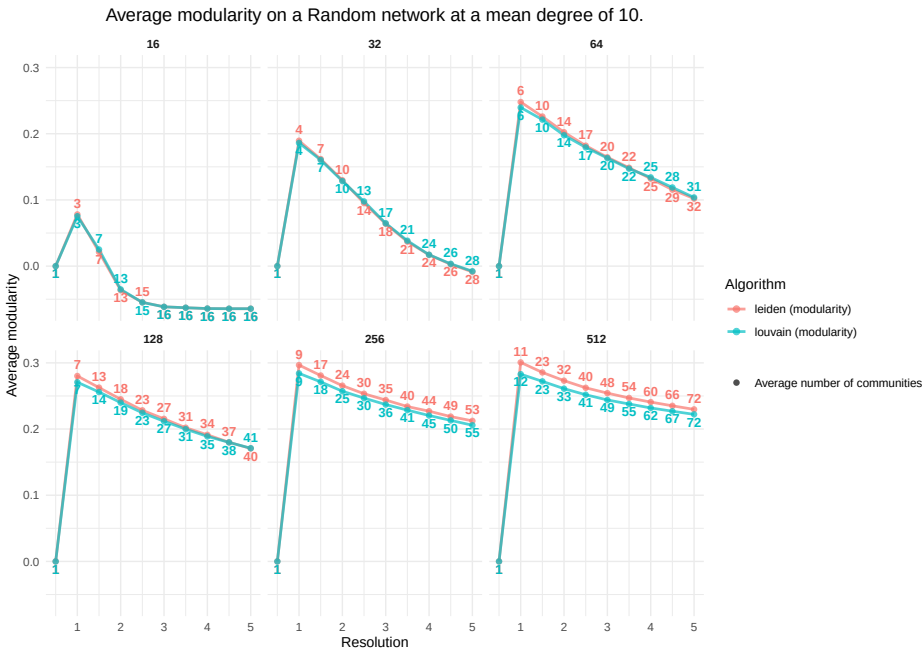


FIGURE A.4: Average modularity results for random networks at a mean degree of ten. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

Conductance

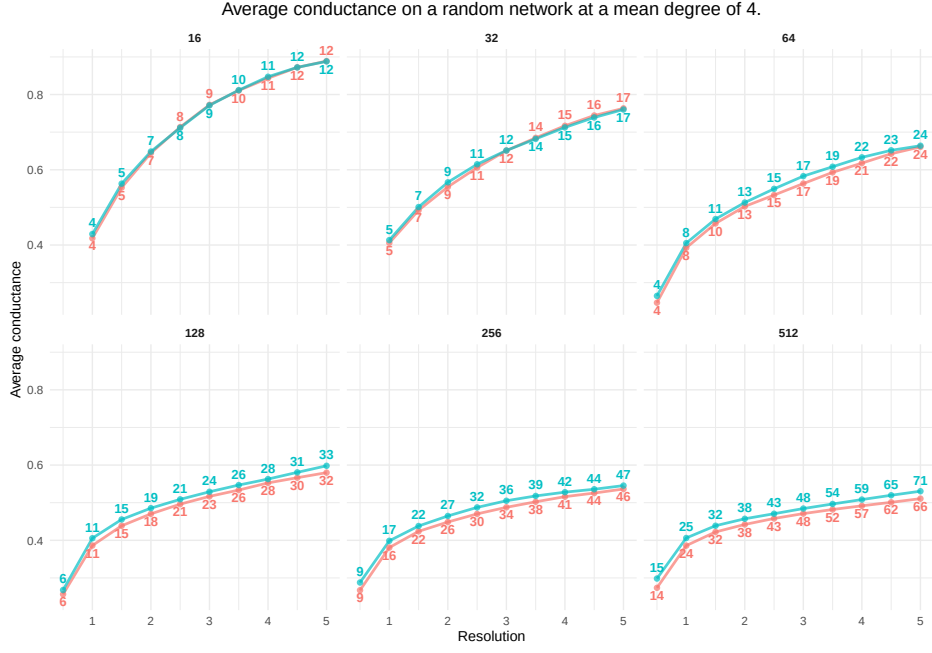


FIGURE A.5: Average conductance results for random networks at a mean degree of four. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

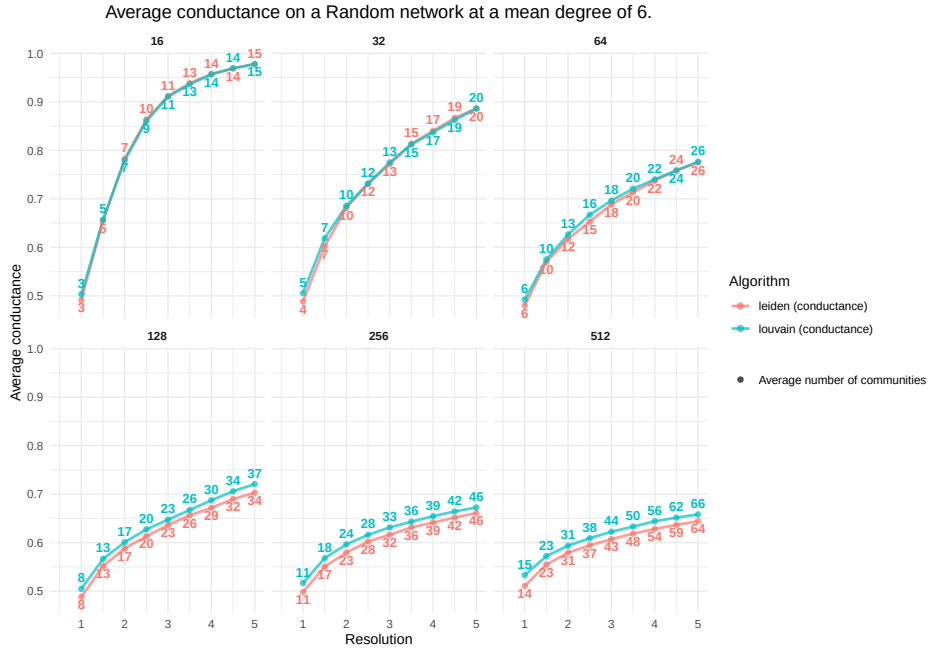


FIGURE A.6: Average conductance results for random networks at a mean degree of six. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

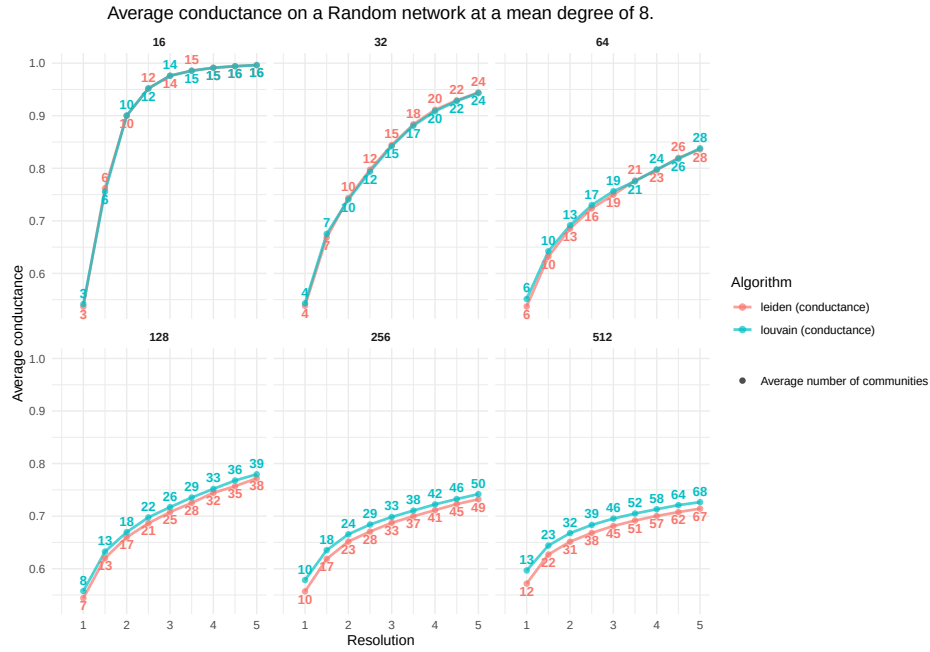


FIGURE A.7: Average conductance results for random networks at a mean degree of eight. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

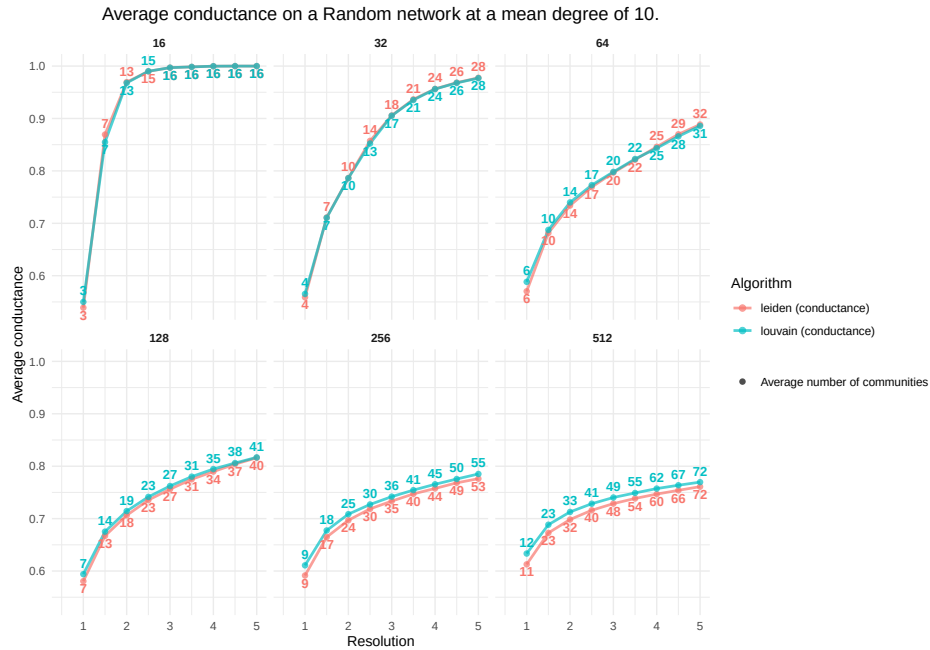


FIGURE A.8: Average conductance results for random networks at a mean degree of ten. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

Stability

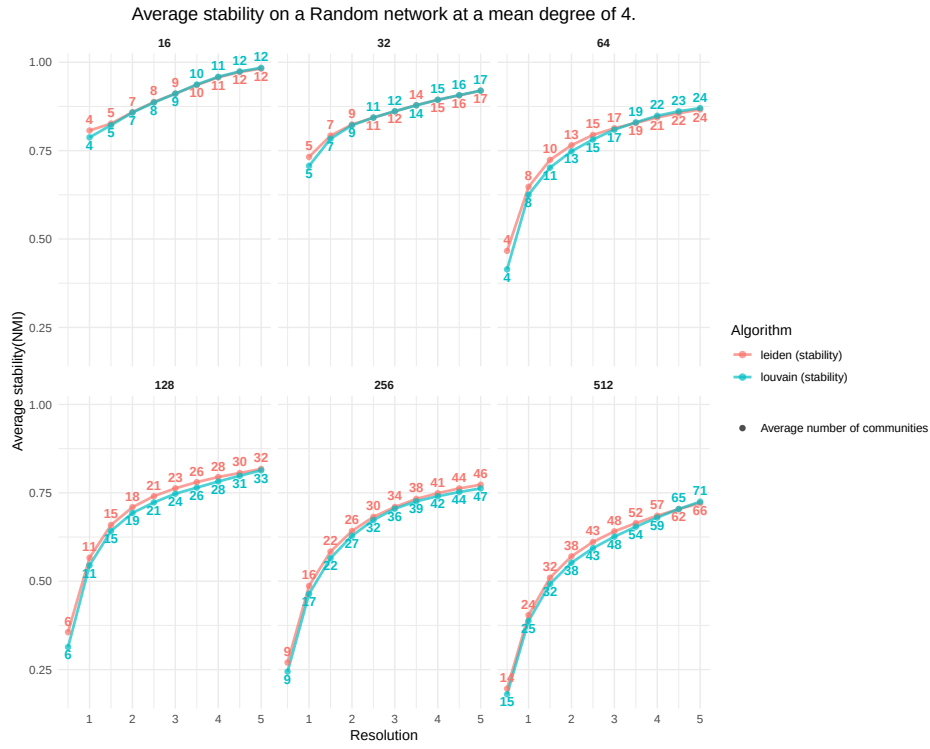


FIGURE A.9: Average stability results for random networks at a mean degree of four. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

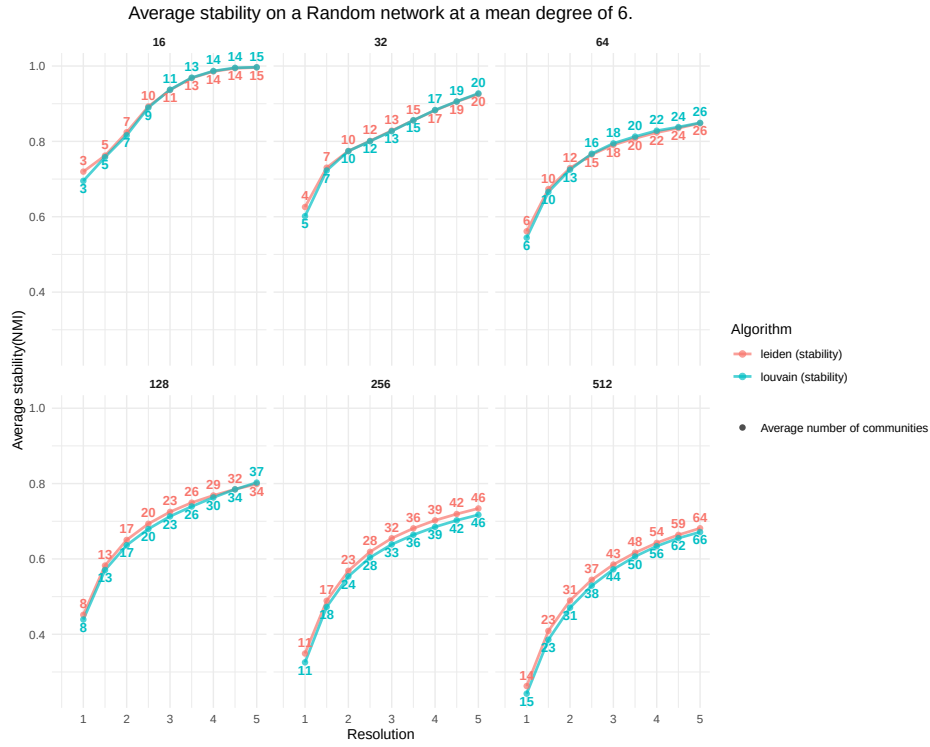


FIGURE A.10: Average stability results for random networks at a mean degree of six. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

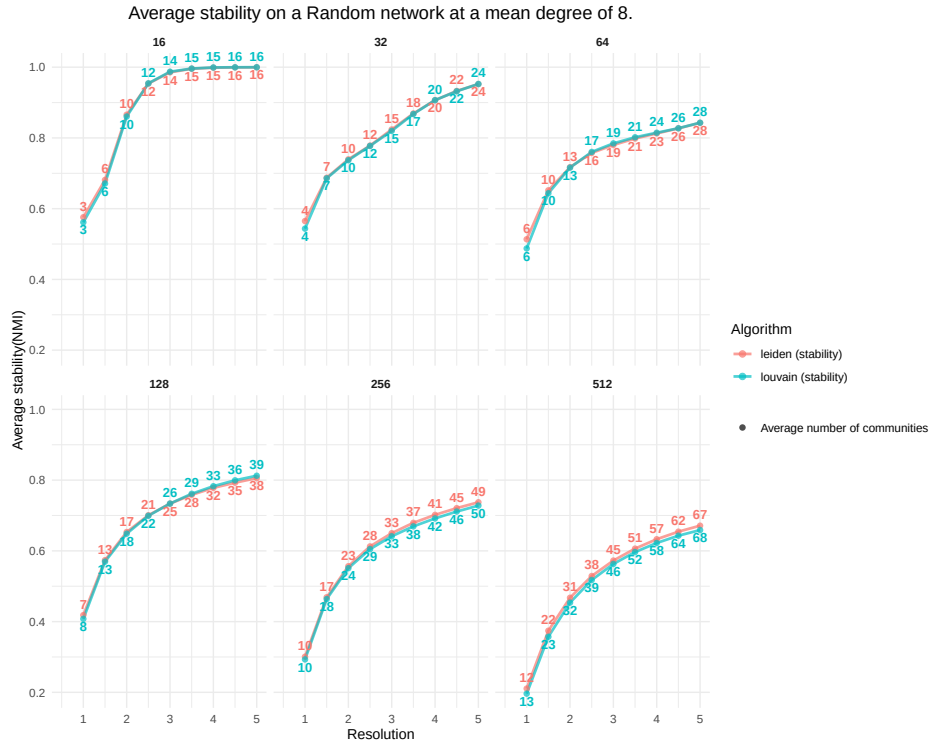


FIGURE A.11: Average stability results for random networks at a mean degree of eight. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

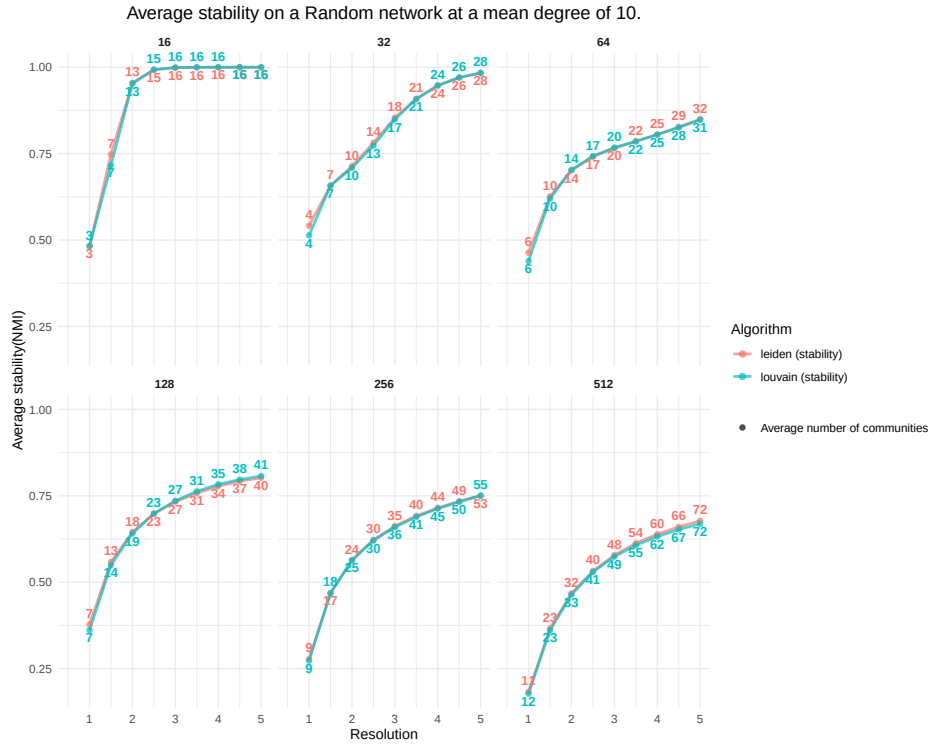


FIGURE A.12: Average stability results for random networks at a mean degree of ten. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

A.2 Small-world networks at a fixed mean degree

Modularity

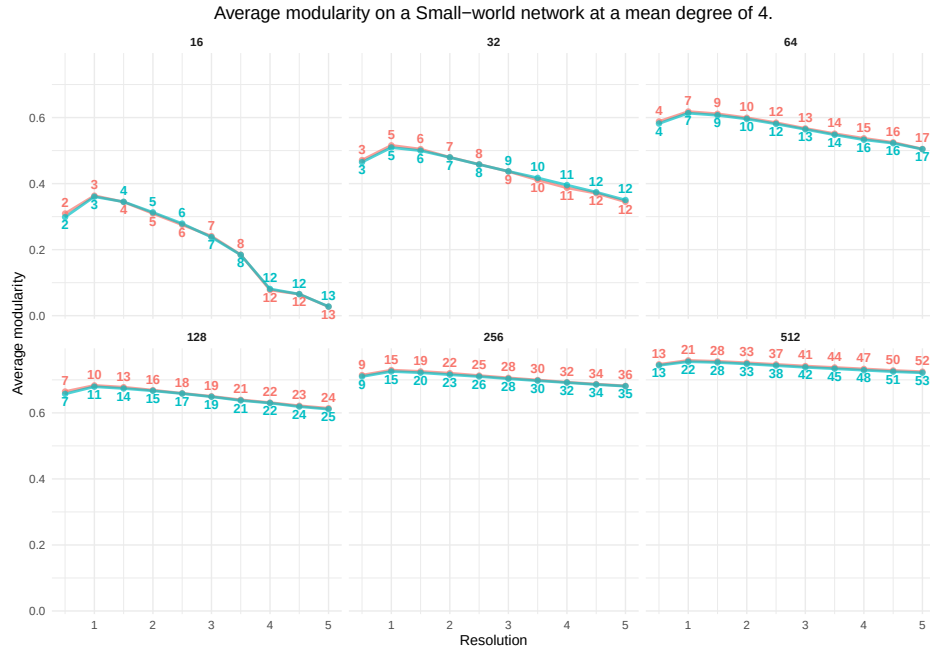


FIGURE A.13: Average modularity results for small-world networks at a mean degree of four. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

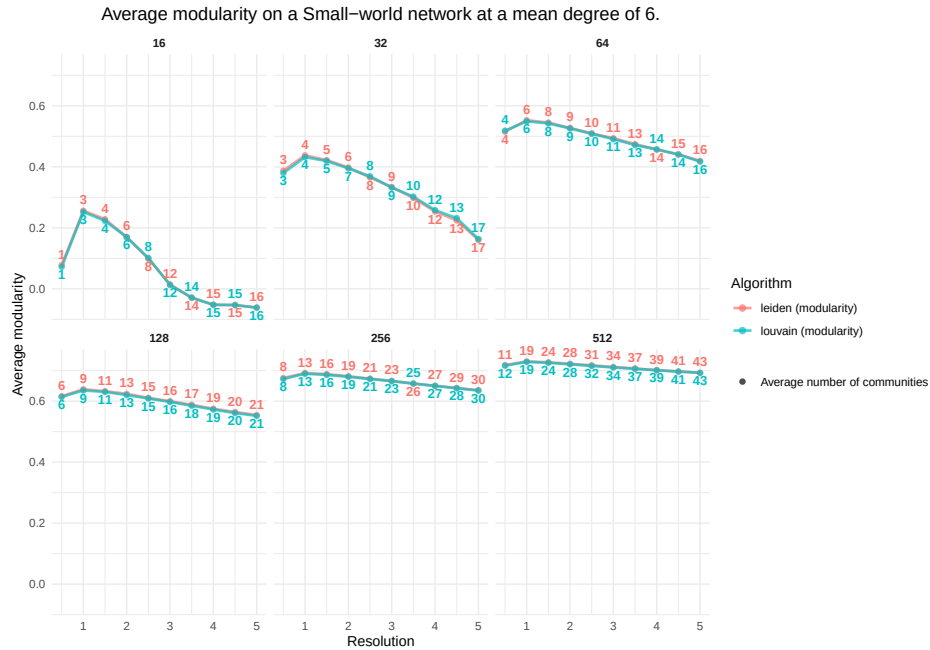


FIGURE A.14: Average modularity results for small-world networks at a mean degree of six. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

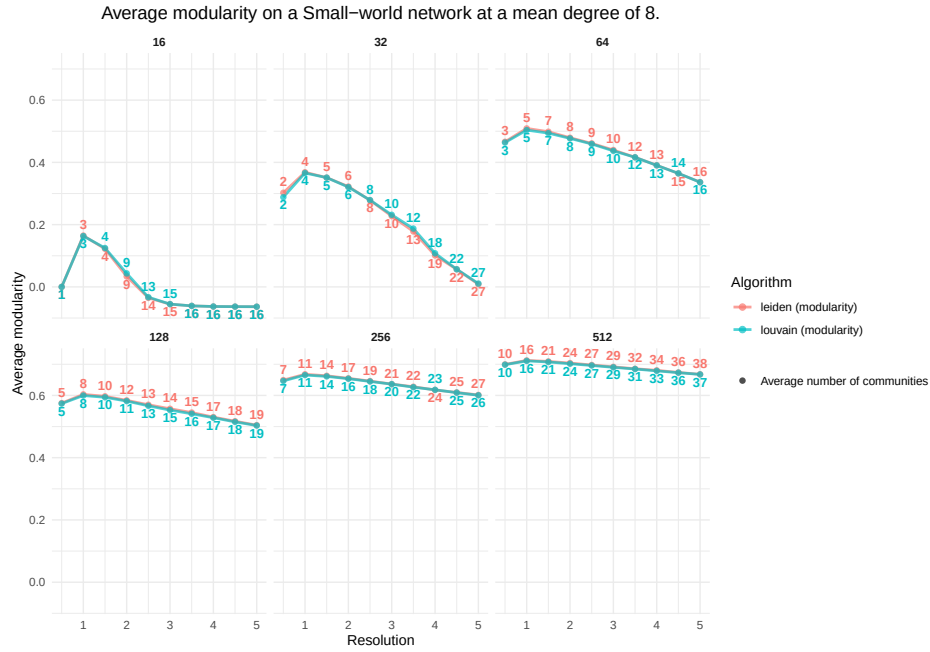


FIGURE A.15: Average modularity results for small-world networks at a mean degree of eight. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

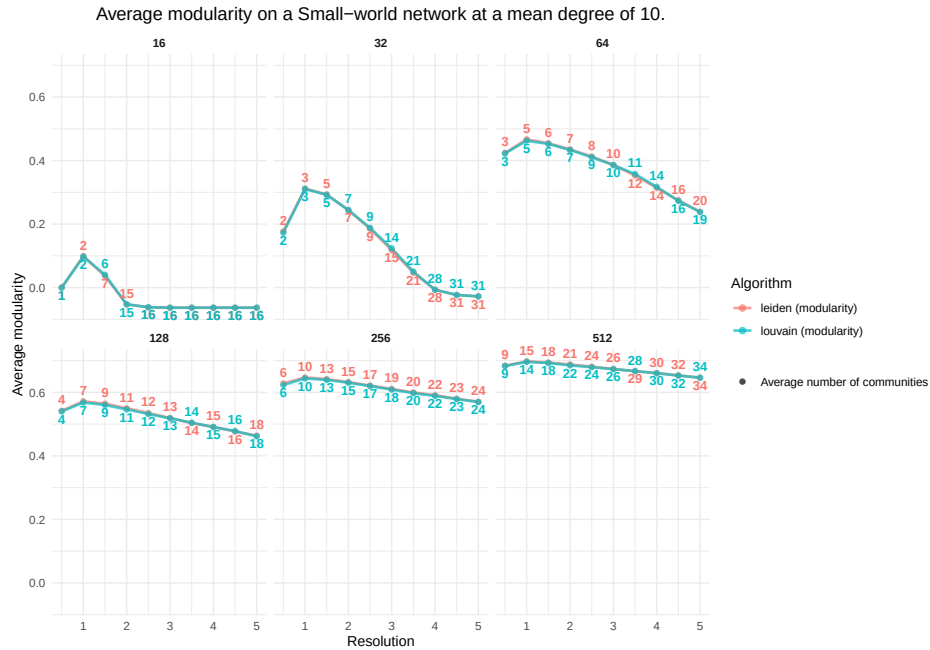


FIGURE A.16: Average modularity results for small-world networks at a mean degree of ten. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

Conductance

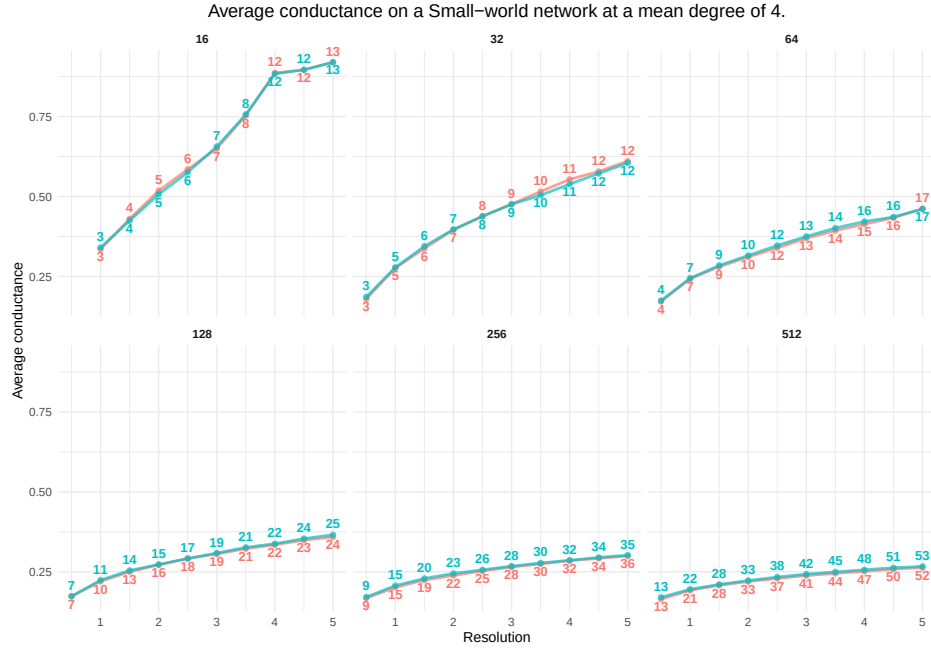


FIGURE A.17: Average conductance results for small-world networks at a mean degree of four. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

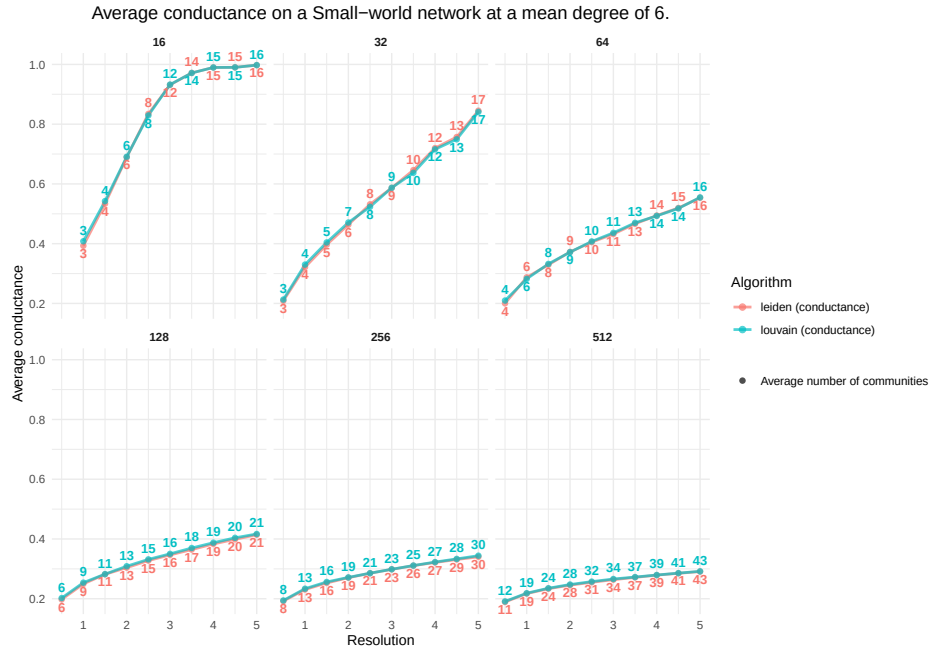


FIGURE A.18: Average conductance results for small-world networks at a mean degree of six. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

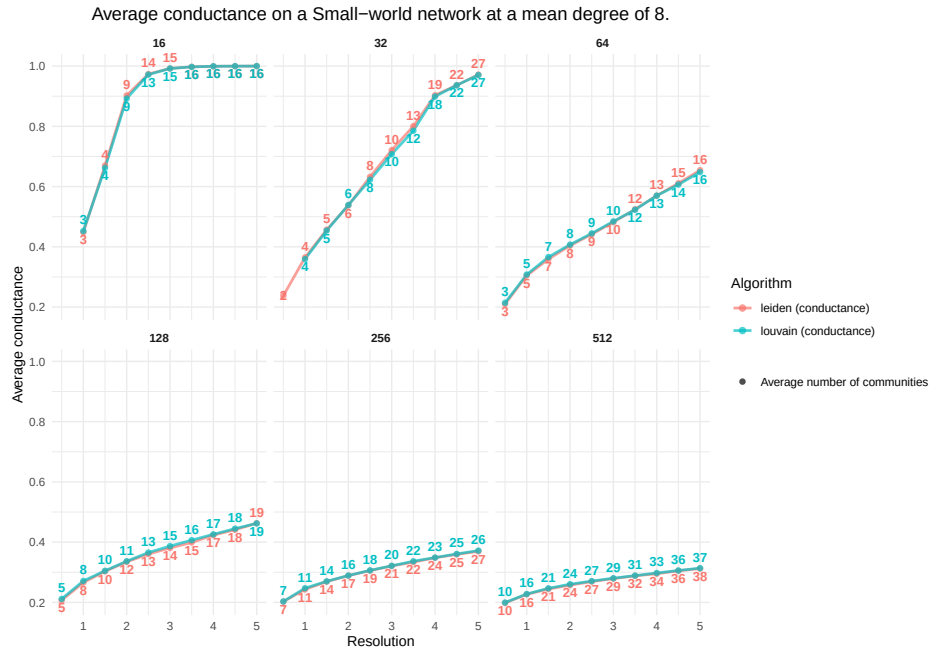


FIGURE A.19: Average conductance results for small-world networks at a mean degree of eight. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

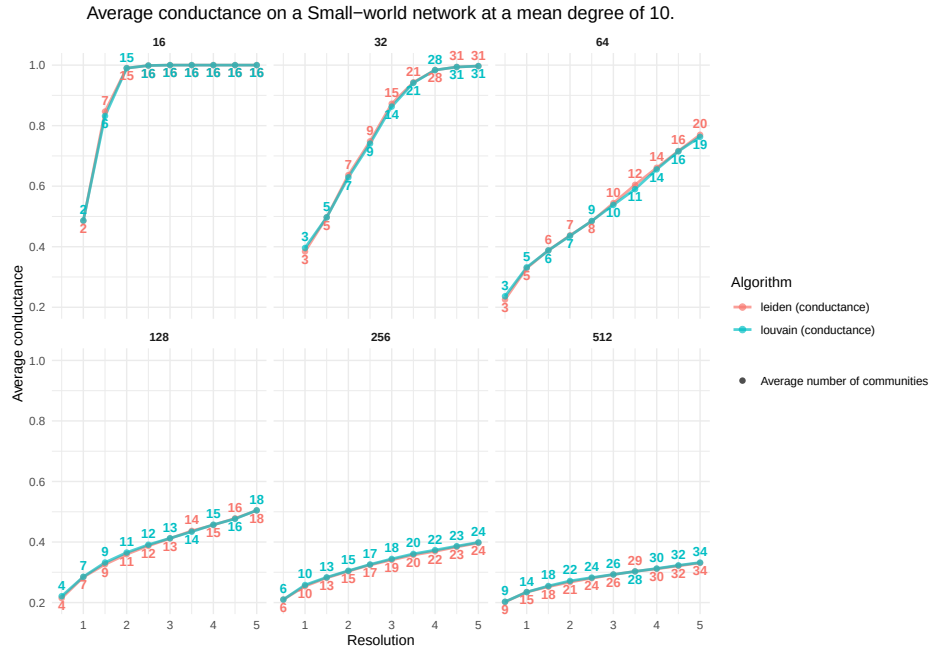


FIGURE A.20: Average conductance results for small-world networks at a mean degree of ten. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

Stability

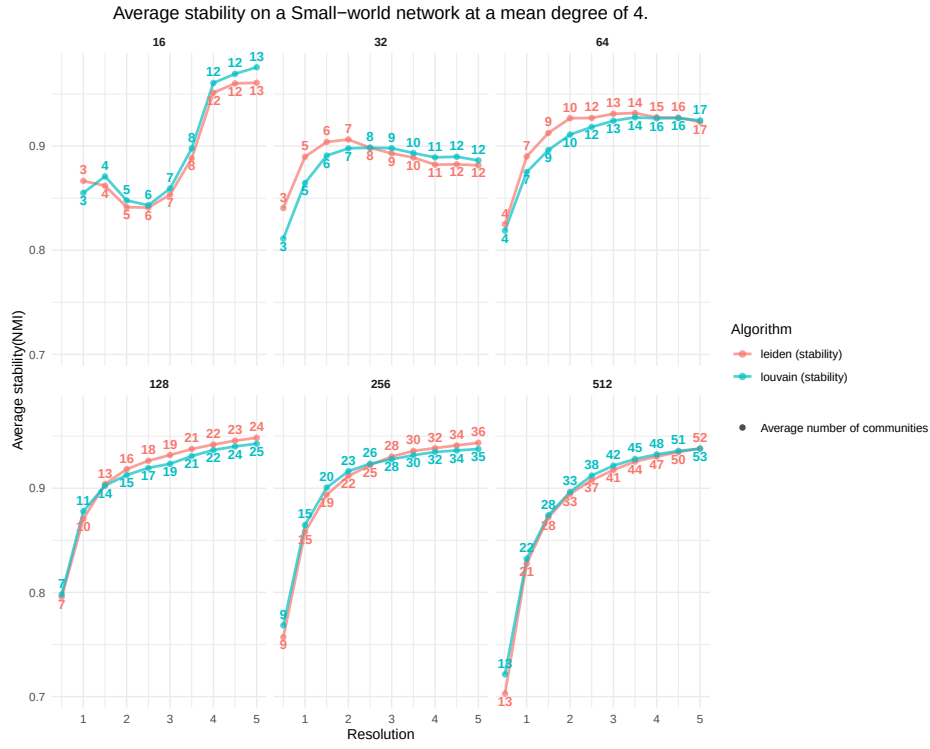


FIGURE A.21: Average stability results for small-world networks at a mean degree of four. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

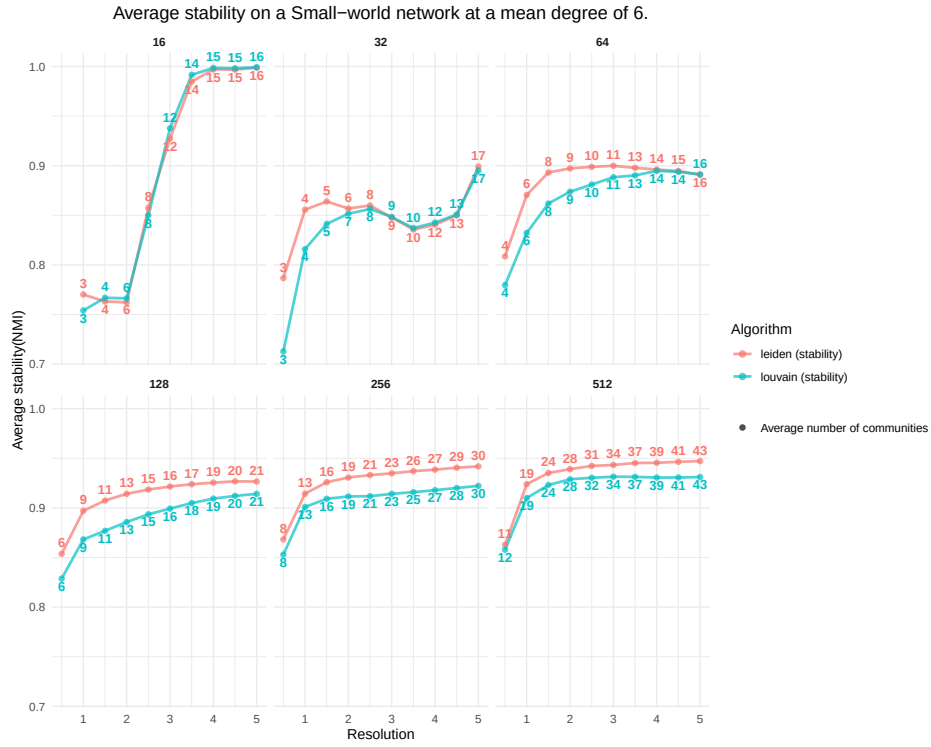


FIGURE A.22: Average stability results for small-world networks at a mean degree of six. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

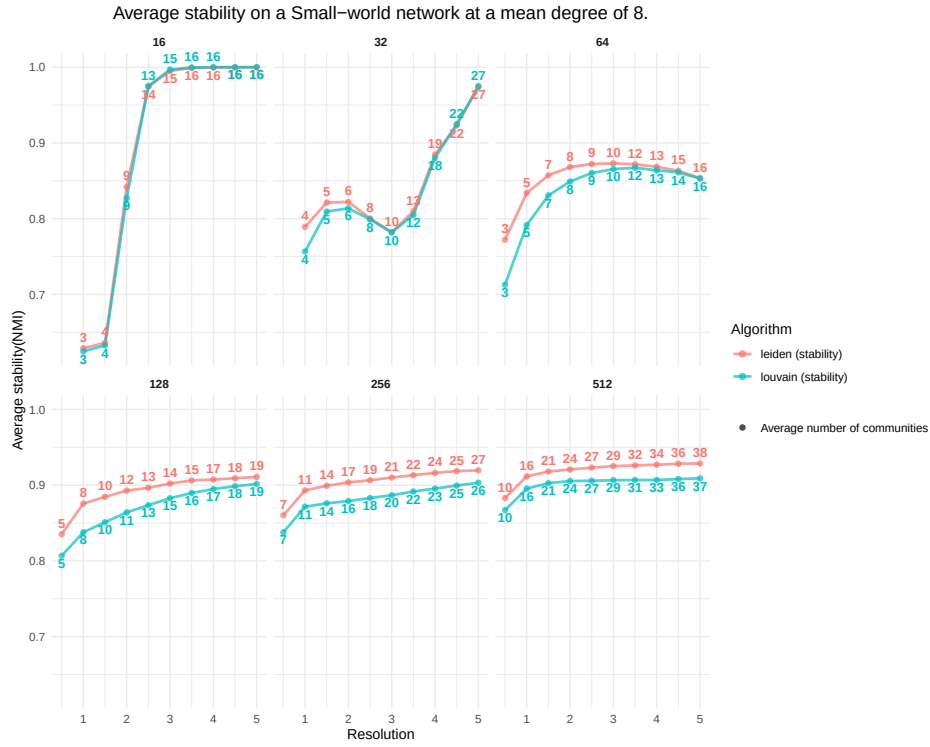


FIGURE A.23: Average stability results for small-world networks at a mean degree of eight. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

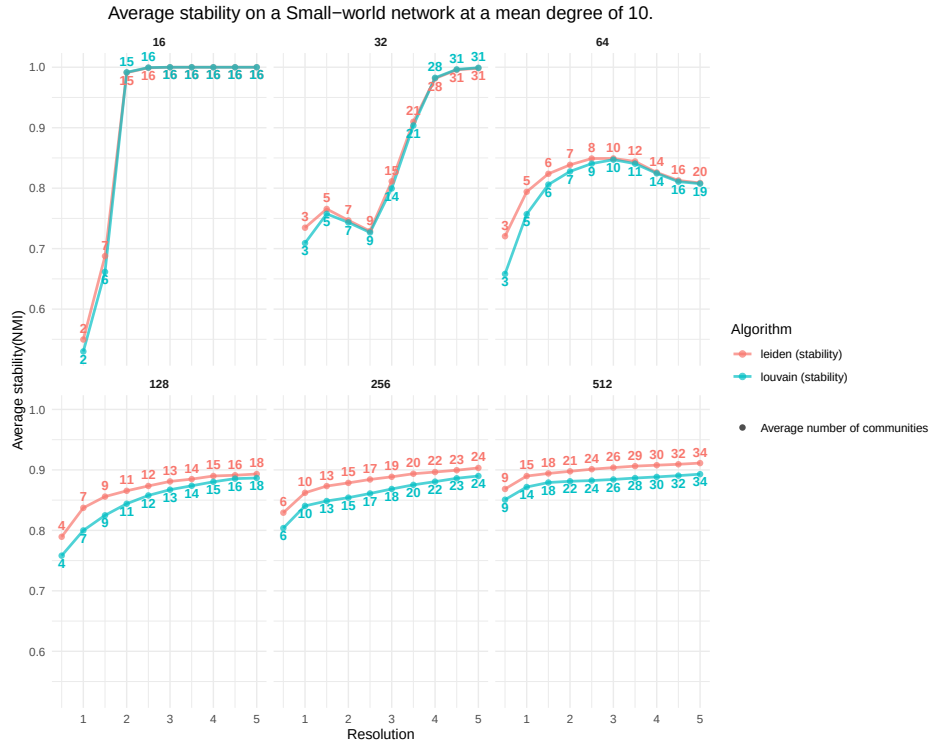


FIGURE A.24: Average stability results for small-world networks at a mean degree of ten. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

A.3 Scale-free networks at a fixed mean degree

Modularity

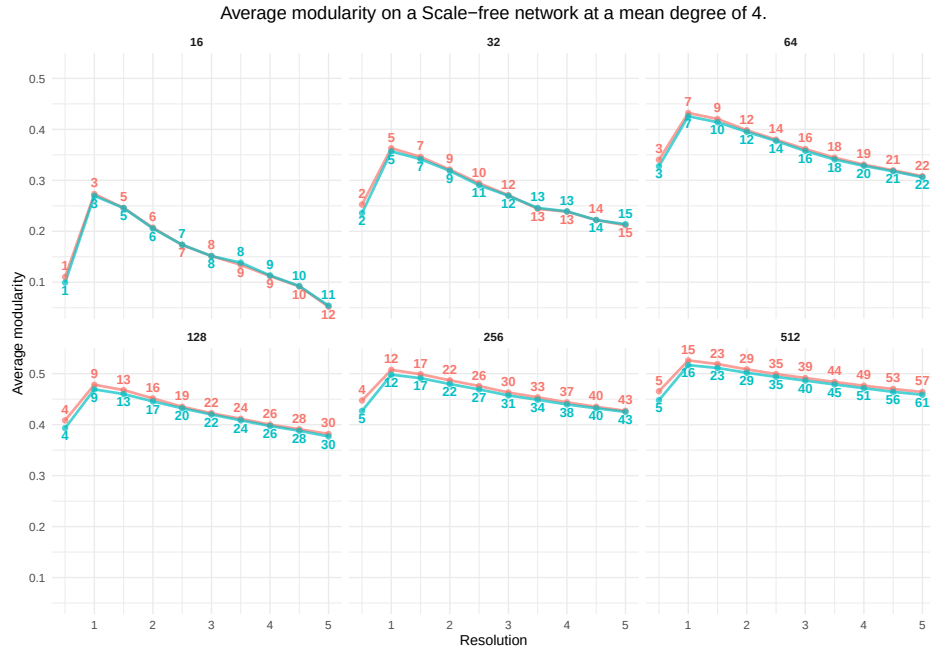


FIGURE A.25: Average modularity results for scale-free networks at a mean degree of four. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

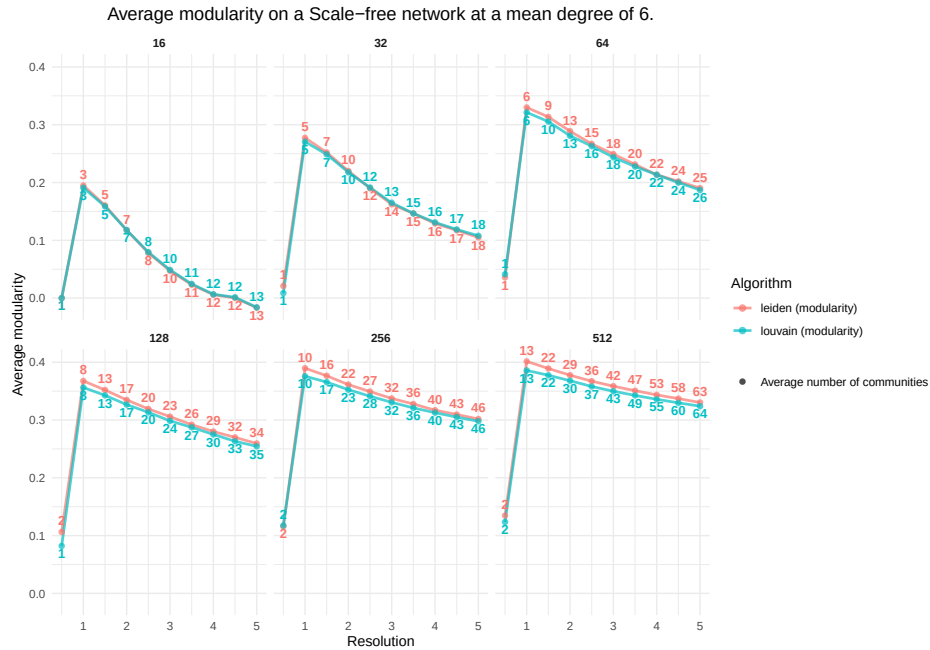


FIGURE A.26: Average modularity results for scale-free networks at a mean degree of six. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

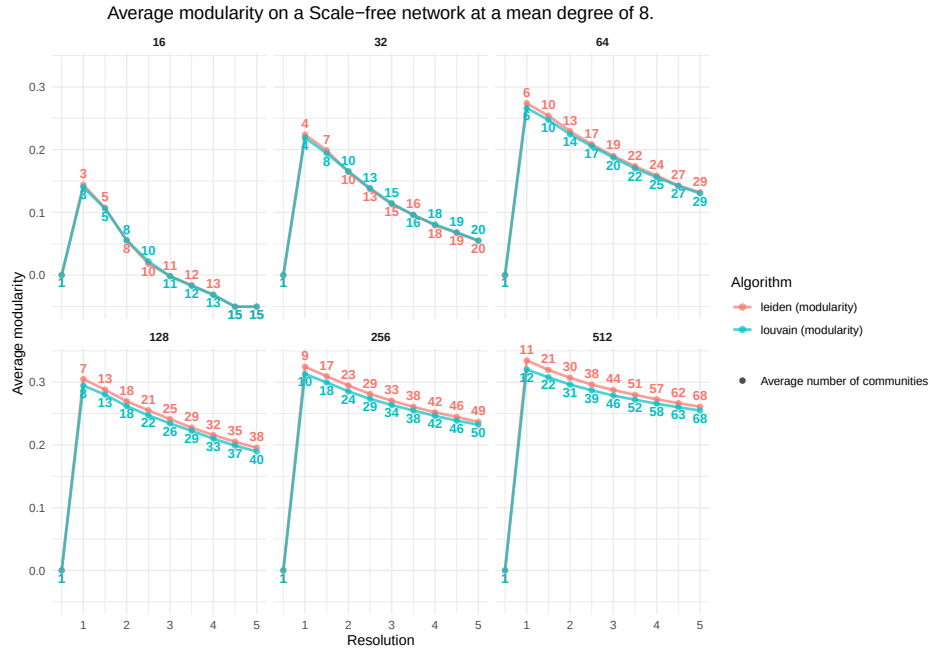


FIGURE A.27: Average modularity results for scale-free networks at a mean degree of eight. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

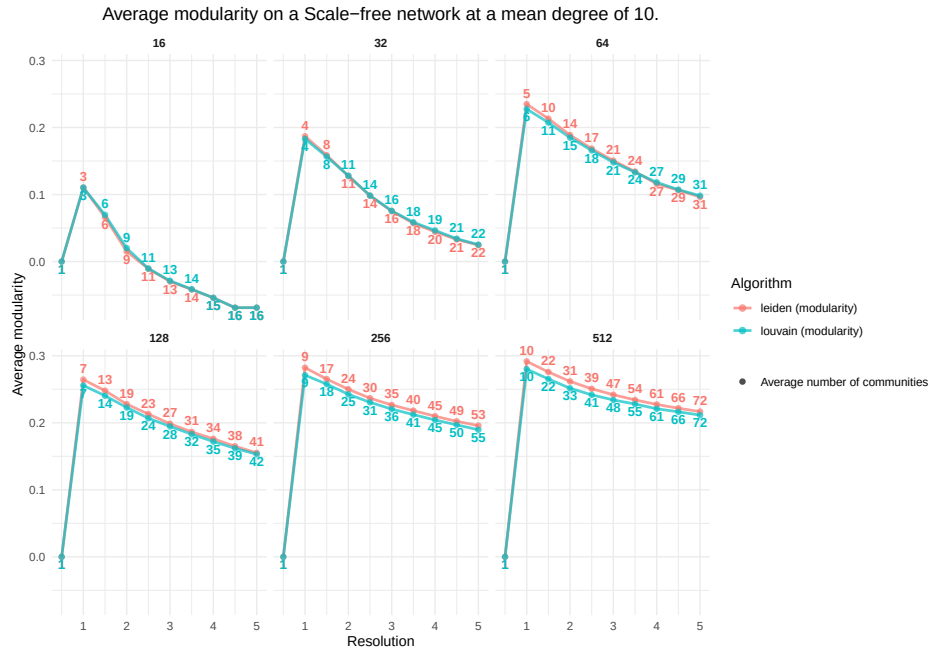


FIGURE A.28: Average modularity results for scale-free networks at a mean degree of ten. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

Conductance

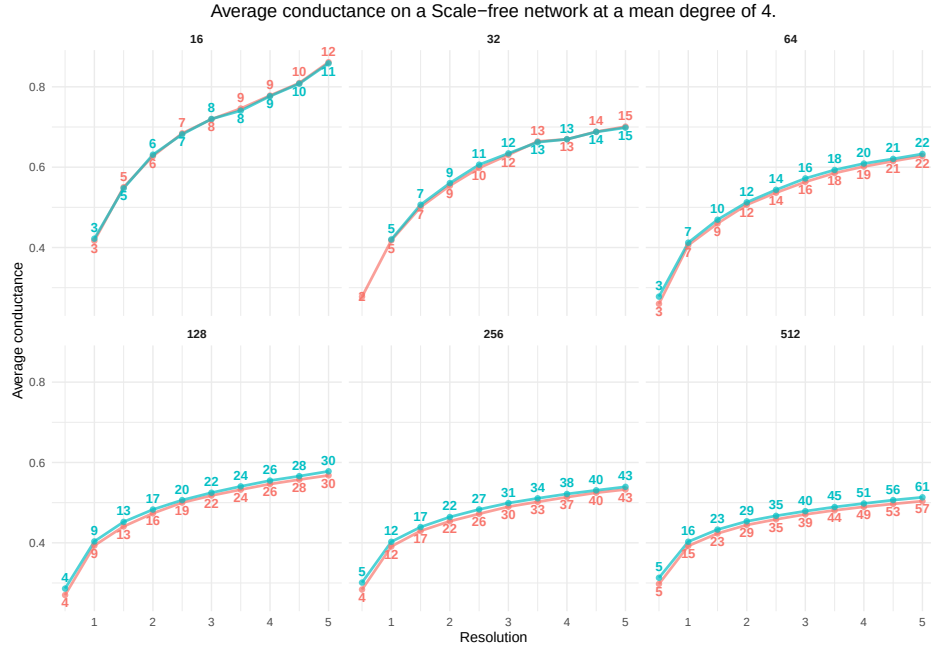


FIGURE A.29: Average conductance results for scale-free networks at a mean degree of four. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

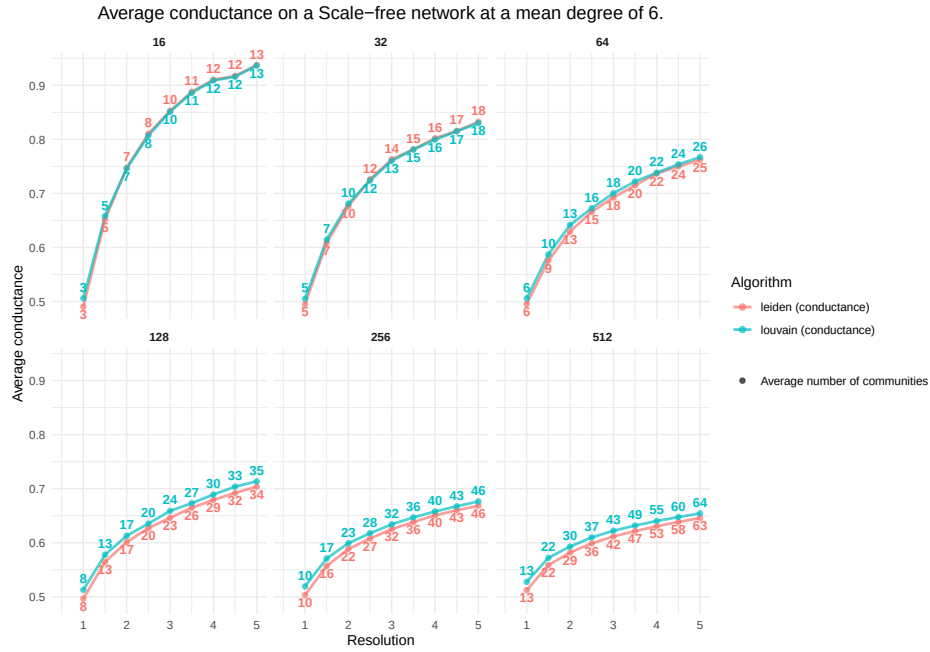


FIGURE A.30: Average conductance results for scale-free networks at a mean degree of six. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

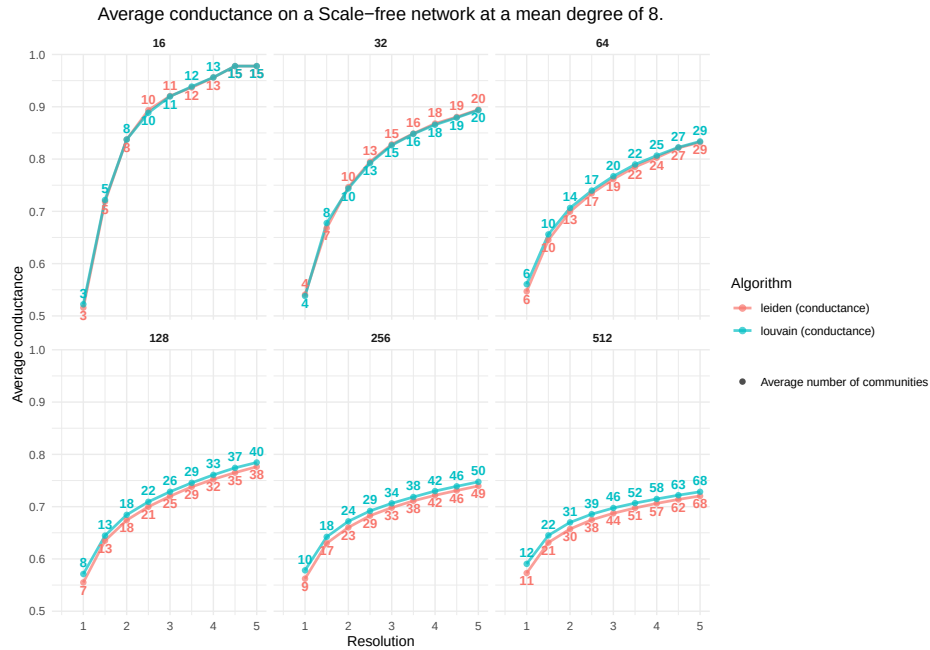


FIGURE A.31: Average conductance results for scale-free networks at a mean degree of eight. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

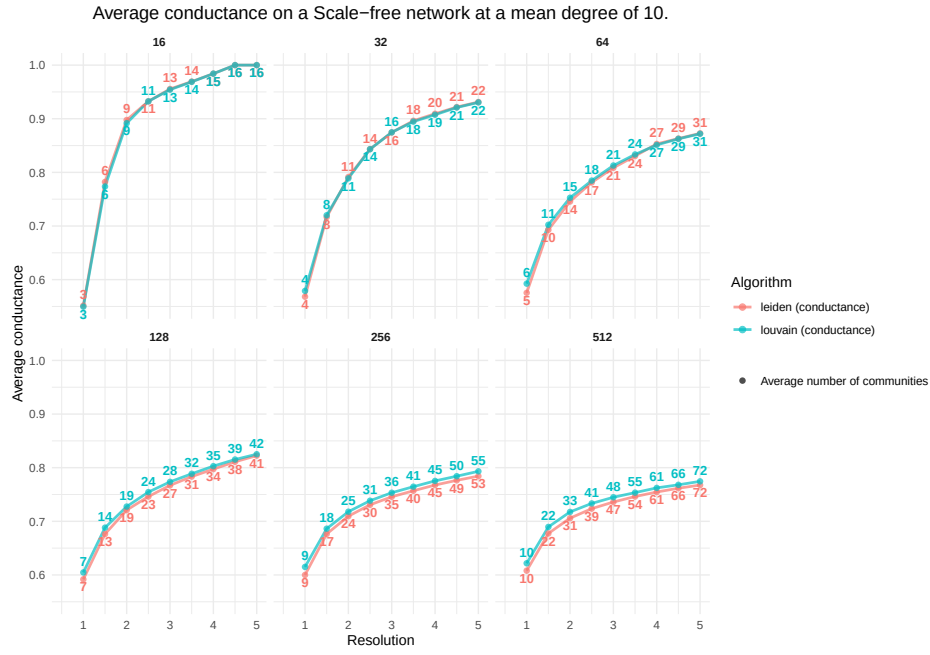


FIGURE A.32: Average conductance results for scale-free networks at a mean degree of ten. Each point represents the mean of 100 runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

Stability

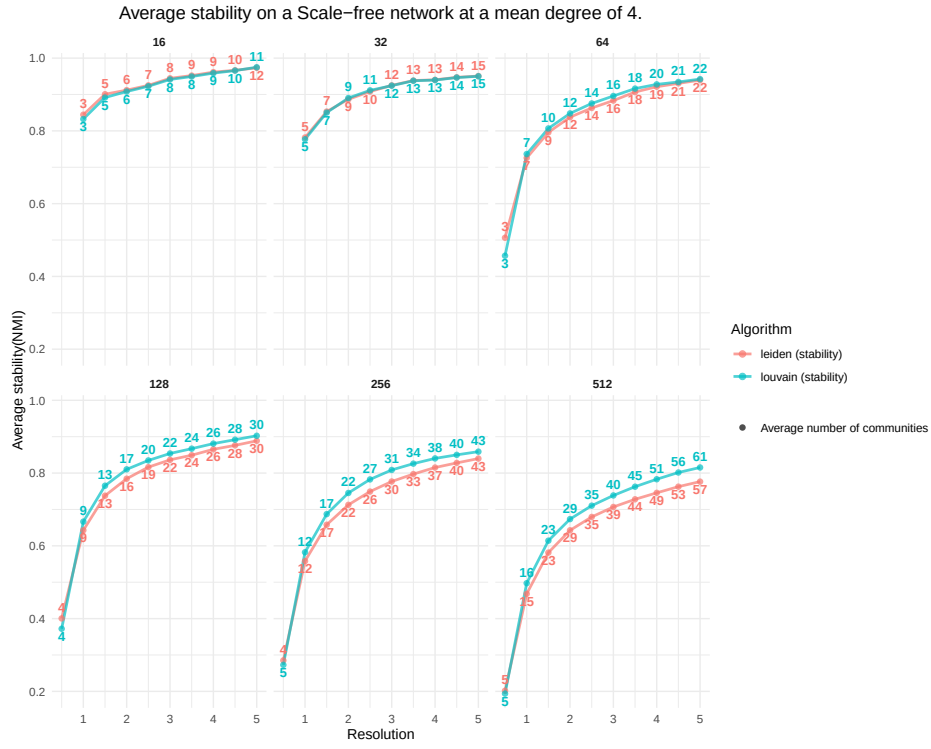


FIGURE A.33: Average stability results for scale-free networks at a mean degree of four. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

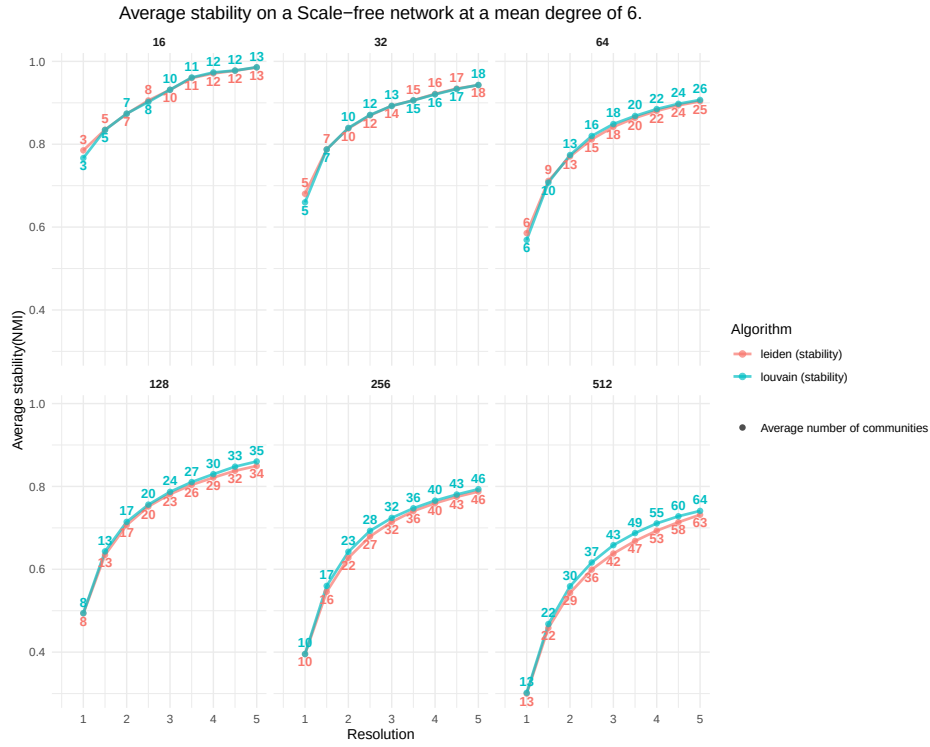


FIGURE A.34: Average stability results for scale-free networks at a mean degree of six. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

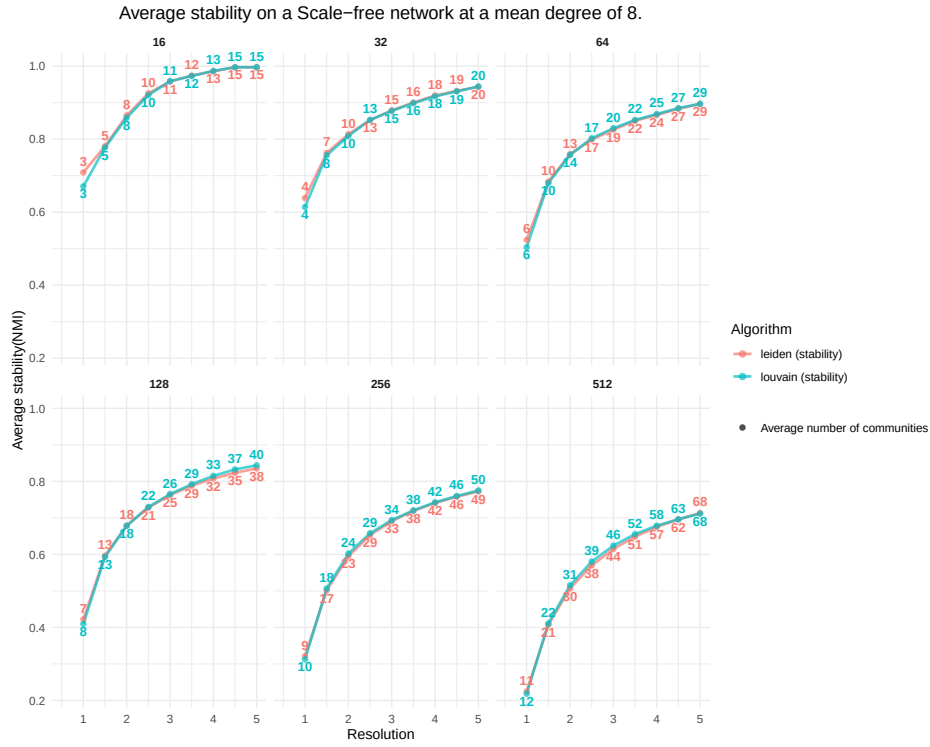


FIGURE A.35: Average stability results for scale-free networks at a mean degree of eight. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

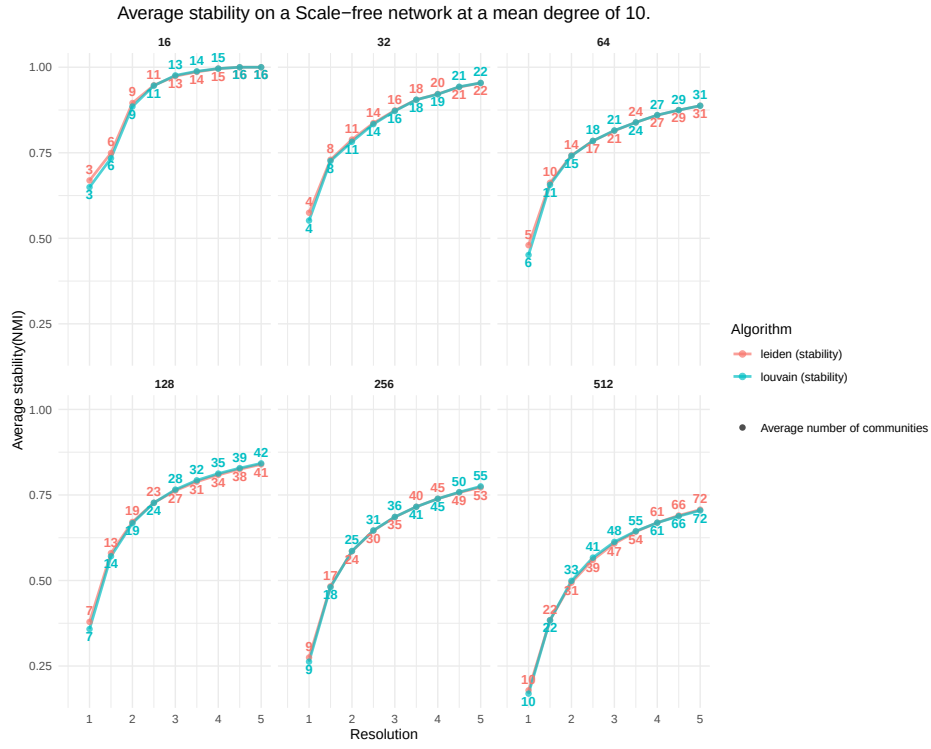


FIGURE A.36: Average stability results for scale-free networks at a mean degree of ten. In each of the 100 runs, stability was calculated 50 times and averaged within the run; these averaged values were then averaged across all runs. The blue and orange lines correspond to the Louvain and Leiden algorithms, respectively. The numbers above the points indicate the average number of communities detected, the bold numbers denote network sizes, and the numbers along the x-axis represent the resolution values.

Bibliography

- Albert, Réka and Albert-László Barabási (2002). “Statistical mechanics of complex networks”. In: *Rev. Mod. Phys.* 74 (1), pp. 47–97. DOI: [10.1103/RevModPhys.74.47](https://doi.org/10.1103/RevModPhys.74.47). URL: <https://link.aps.org/doi/10.1103/RevModPhys.74.47>.
- Anandkumar, Animashree et al. (2014). “A Tensor Approach to Learning Mixed Membership Community Models”. In: *Journal of Machine Learning Research* 15.1, pp. 2239–2312. URL: <http://jmlr.org/papers/v15/anandkumar14b.html>.
- Bae, S. et al. (2017). “Scalable and Efficient Flow-Based Community Detection for Large-Scale Graph Analysis”. In: *ACM Transactions on Knowledge Discovery from Data* 11.1, pp. 1–30. DOI: [10.1145/2992785](https://doi.org/10.1145/2992785).
- Barabási, Albert-László (2013). “Network Science”. In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 371.1987, p. 20120375. DOI: [10.1098/rsta.2012.0375](https://doi.org/10.1098/rsta.2012.0375). eprint: <https://royalsocietypublishing.org/doi/pdf/10.1098/rsta.2012.0375>. URL: <https://doi.org/10.1098/rsta.2012.0375>.
- Barabási, Albert-László and Réka Albert (1999). “Emergence of Scaling in Random Networks (1999)”. In: *Science* 286.5439, pp. 509–512. URL: <https://www.science.org/doi/10.1126/science.286.5439.509>.
- Blondel, Vincent D. et al. (2008). “Fast unfolding of communities in large networks”. In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10, P10008. DOI: [10.1088/1742-5468/2008/10/P10008](https://doi.org/10.1088/1742-5468/2008/10/P10008).
- Bollobás, Béla (2001). *Random Graphs (2001)*. Vol. 73. Cambridge, U.K.: Cambridge University Press. ISBN: 9780521797221. URL: <https://www.cambridge.org/core/books/random-graphs/67E36A1D0B0DE2F7F234B02A78C36885>.
- Börner, Katy, Soma Sanyal, Alessandro Vespignani, et al. (2007). “Network science”. In: *Annu. rev. inf. sci. technol.* 41.1, pp. 537–607.
- Brandes, U. (2005). *Network Analysis: Methodological Foundations*. Springer Science & Business Media.
- Brandes, Ulrik et al. (2013). “What is network science?” In: *Network Science* 1.1, pp. 1–15. DOI: [10.1017/nws.2013.2](https://doi.org/10.1017/nws.2013.2). URL: <https://doi.org/10.1017/nws.2013.2>.
- Chiquet, Julien, Guillem Rigai, and Martina Sundqvist (2023). *aricode: Efficient Computations of Standard Clustering Comparison Measures*. R package version 1.0.3. URL: <https://CRAN.R-project.org/package=aricode>.

- Christensen, A. P. et al. (2024). “Comparing community detection algorithms in psychometric networks: A Monte Carlo simulation+6”. In: *Behavior Research Methods* 56, pp. 1485–1505. DOI: [10.3758/s13428-023-02106-4](https://doi.org/10.3758/s13428-023-02106-4).
- Christley, RM and NP French (2003). “Small-world topology of UK racing: the potential for rapid spread of infectious agents”. In: *Equine veterinary journal* 35.6, pp. 586–589.
- Csardi, Gabor and Tamas Nepusz (2006). “The igraph software package for complex network research”. In: *InterJournal Complex Systems*, p. 1695. URL: <https://igraph.org>.
- Deitrick, William et al. (2013). “Enhancing sentiment analysis on Twitter using community detection”. In: *Communications and Network* 5, pp. 192–197. DOI: [10.4236/cn.2013.53022](https://doi.org/10.4236/cn.2013.53022).
- Delvenne, JC., SN. Yaliraki, et al. (2010). “Stability of Graph Communities Across Time Scales (2010)”. In: *Proceedings of the National Academy of Sciences of the United States of America* 107.29, pp. 12755–12760. URL: <https://doi.org/10.1073/pnas.0903215107>.
- El Naschie, MS (2004). “Small world network, $\varepsilon(\infty)$ topology and the mass spectrum of high energy particles physics”. In: *Chaos, Solitons & Fractals* 19.3, pp. 689–697.
- Emirbayer, Mustafa and Jeff Goodwin (1994). “Network Analysis, Culture, and the Problem of Agency”. In: *American Journal of Sociology* 99.6, pp. 1411–1454. DOI: [10.1086/230450](https://doi.org/10.1086/230450). URL: <http://www.jstor.org/stable/2782580>.
- Erdős, Paul and Alfréd Rényi (1960). “On the Evolution of Random Graphs”. In: *Publications of the Mathematical Institute of the Hungarian Academy of Sciences* 5, pp. 17–61.
- Erdős, Paul and Alfréd Rényi (1959). “On Random Graphs I”. In: *Publicationes Mathematicae Debrecen* 6, pp. 290–297. URL: <https://erdoscenter.renyi.hu/sites/default/files/publikaciok/erdos1959onrandom.pdf>.
- Farago, Andras (2009). “Scalability of node degrees in random wireless network topologies”. In: *IEEE Journal on Selected Areas in Communications* 27.7, pp. 1238–1244. DOI: [10.1109/JSAC.2009.090919](https://doi.org/10.1109/JSAC.2009.090919).
- Faragó, András (2012). “Network Topology Models for Multihop Wireless Networks”. In: *ISRN Communications and Networking*. DOI: [10.5402/2012/362603](https://doi.org/10.5402/2012/362603). URL: <https://doi.org/10.5402/2012/362603>.
- Fenn, D. J. et al. (2009). “Dynamical Clustering of Exchange Rates (2009)”. In: *Quantitative Finance* 11.4, pp. 587–605. URL: <https://doi.org/10.1080/14697680902995848>.
- Fortunato, S. and H. Darko (2016). “Community detection in networks: A user guide”. In: *Physics Reports* 659, pp. 1–44. eprint: <https://www.sciencedirect.com/science/article/pii/S0370157316302964>. URL: <https://doi.org/10.1016/j.physrep.2016.09.002>.
- Fortunato, Santo and Marc Barthélemy (2007). “Resolution limit in community detection”. In: *Proceedings of the National Academy of Sciences* 104.1, pp. 36–41. DOI: [10.1073/pnas.0605965104](https://doi.org/10.1073/pnas.0605965104).
- Greaves, GN and EA Davis (1974). “A continuous random network model with three-fold coordination”. In: *Philosophical Magazine* 29.5, pp. 1201–1206.

- Hafner-Burton, Emilie M., Miles Kahler, and Alexander H. Montgomery (2009). "Network Analysis for International Relations". In: *International Organization* 63.3, pp. 559–592. DOI: [10.1017/S0020818309090190](https://doi.org/10.1017/S0020818309090190). URL: <https://doi.org/10.1017/S0020818309090190>.
- Hairol Anuar, Siti Haryanti et al. (2021). "Comparison between Louvain and Leiden Algorithm for Network Structure: A Review". In: *Journal of Physics: Conference Series* 2129.1, p. 012028. DOI: [10.1088/1742-6596/2129/1/012028](https://doi.org/10.1088/1742-6596/2129/1/012028).
- Hairol Anuar, Siti Haryanti et al. (2024). "Identifying Communities with Modularity Metric Using Louvain and Leiden Algorithms". In: *Pertanika Journal of Science & Technology* 33.3.
- Hevey, D. (2018). "Network Analysis: A Brief Overview and Tutorial". In: *Health Psychology and Behavioral Medicine* 6.1, pp. 301–328. DOI: [10.1080/21642850.2018.1521283](https://doi.org/10.1080/21642850.2018.1521283). URL: <https://doi.org/10.1080/21642850.2018.1521283>.
- Hong, Seok-Hee et al. (2021). "Louvain-based multi-level graph drawing". In: *Proceedings of the 2021 IEEE 14th Pacific Visualization Symposium (PacificVis)*. IEEE, pp. 151–155. DOI: [10.1109/PacificVis52677.2021.00028](https://doi.org/10.1109/PacificVis52677.2021.00028).
- Horvath, Steve and Jun Dong (2008). "Geometric Interpretation of Gene Coexpression Network Analysis". In: *PLoS Computational Biology* 4.8, e1000117. DOI: [10.1371/journal.pcbi.1000117](https://doi.org/10.1371/journal.pcbi.1000117). URL: <https://doi.org/10.1371/journal.pcbi.1000117>.
- Jeub, Lucas G. S. et al. (2015). "Think Locally, Act Locally: The Detection of Small, Medium-Sized, and Large Communities in Large Networks". In: *Physical Review E* 91.1, p. 012821. DOI: [10.1103/PhysRevE.91.012821](https://doi.org/10.1103/PhysRevE.91.012821). URL: <https://doi.org/10.1103/PhysRevE.91.012821>.
- Jin, Di et al. (2023). "A Survey of Community Detection Approaches: From Statistical Modeling to Deep Learning". In: *IEEE Transactions on Knowledge and Data Engineering* 35.2, pp. 1149–1170. DOI: [10.1109/TKDE.2021.3104155](https://doi.org/10.1109/TKDE.2021.3104155).
- Khattoon, Mehjabin and W. Aisha Banu (2021). "A survey on community detection methods in social networks". In: *Materials Today: Proceedings* 45, pp. 2897–2903. DOI: [10.1016/j.matpr.2020.11.726](https://doi.org/10.1016/j.matpr.2020.11.726).
- Learned-Miller, Erik G. (Sept. 2013). *Entropy and Mutual Information*. Tech. rep. <https://people.cs.umass.edu/~elm/papers/entropy.pdf>. Amherst, MA: Department of Computer Science, University of Massachusetts, Amherst.
- Lewis, Ted G. (2009). *Network Science: Theory and Applications*. John Wiley & Sons.
- Liu, Xin, Hui-Min Cheng, and Zhong-Yuan Zhang (2020). "Evaluation of Community Detection Methods". In: *IEEE Transactions on Knowledge and Data Engineering* 32.9, pp. 1736–1746. DOI: [10.1109/TKDE.2019.2911943](https://doi.org/10.1109/TKDE.2019.2911943).
- Lu, Hao, Mahantesh Halappanavar, and Ananth Kalyanaraman (2015). "Parallel heuristics for scalable community detection". In: *Parallel Computing* 46, pp. 50–67. DOI: [10.1016/j.parco.2015.03.003](https://doi.org/10.1016/j.parco.2015.03.003).
- Lynn, Christopher W, Caroline M Holmes, and Stephanie E Palmer (June 2024). "Emergent scale-free networks". In: *PNAS Nexus* 3.7, pgae236. ISSN: 2752-6542. DOI: [10.1093/pnasnexus/pgae236](https://doi.org/10.1093/pnasnexus/pgae236). eprint: <https://academic.oup.com/pnasnexus/article-pdf/>

- 3/7/pgae236/58645766/pgae236.pdf. URL: <https://doi.org/10.1093/pnasnexus/pgae236>.
- Mohammadi, Mehdi, Mohammad Fazlali, and Mohammad Hosseinzadeh (2021). “Accelerating Louvain community detection algorithm on graphic processing unit”. In: *Journal of Supercomputing* 77, pp. 6056–6077. DOI: [10.1007/s11227-020-03510-9](https://doi.org/10.1007/s11227-020-03510-9).
- Mucha, Peter J. et al. (2010). “Community Structure in Time-Dependent, Multiscale, and Multiplex Networks”. In: *Science* 328.5980, pp. 876–878. DOI: [10.1126/science.1184819](https://doi.org/10.1126/science.1184819). eprint: <https://www.science.org/doi/pdf/10.1126/science.1184819>. URL: <https://doi.org/10.1126/science.1184819>.
- Natarajan, M. (2016). *Community Detection Algorithms*. Lecture slides. Accessed: 2025-07-01. URL: <https://www.jsums.edu/nmeghanathan/files/2016/01/CSC641-Sp2016-Module-4-Community1.pdf>.
- Newman, M. E. J. and M. Girvan (Feb. 2004). “Finding and evaluating community structure in networks”. In: *Physical Review E* 69.2. ISSN: 1550-2376. DOI: [10.1103/PhysRevE.69.026113](https://doi.org/10.1103/PhysRevE.69.026113). URL: <http://dx.doi.org/10.1103/PhysRevE.69.026113>.
- Ostovari, Mina and Denny Yu (2019). “Impact of care provider network characteristics on patient outcomes: Usage of social network analysis and a multi-scale community detection”. In: *PLOS ONE* 14.9, e0222016. DOI: [10.1371/journal.pone.0222016](https://doi.org/10.1371/journal.pone.0222016).
- Ozaki, N., H. Tezuka, and M. Inaba (2016). “A Simple Acceleration Method for the Louvain Algorithm”. In: *International Journal of Computer and Electrical Engineering* 8, pp. 207–218. DOI: [10.17706/IJCEE.2016.8.3.207-218](https://doi.org/10.17706/IJCEE.2016.8.3.207-218).
- Papadopoulos, Fragkiskos et al. (2010). “Greedy Forwarding in Dynamic Scale-Free Networks Embedded in Hyperbolic Metric Spaces”. In: *2010 Proceedings IEEE INFOCOM*, pp. 1–9. DOI: [10.1109/INFCOM.2010.5462131](https://doi.org/10.1109/INFCOM.2010.5462131).
- Pedersen, Thomas Lin (2024). *ggraph: An Implementation of Grammar of Graphics for Graphs and Networks*. R package version 2.2.1. URL: <https://CRAN.R-project.org/package=ggraph>.
- Porreca, Annamaria, Fabrizio Maturo, and Viviana Ventre (2024). “Fuzzy centrality measures in social network analysis: Theory and application in a university department collaboration network”. In: *International Journal of Approximate Reasoning*. DOI: [10.1016/j.ijar.2024.109319](https://doi.org/10.1016/j.ijar.2024.109319). URL: <https://doi.org/10.1016/j.ijar.2024.109319>.
- Prell, Christina (2012). *Social Network Analysis: History, Theory and Methodology*. London: Sage.
- R Core Team (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria. URL: <https://www.R-project.org/>.
- Reichardt, Jörg and Stefan Bornholdt (2006). “Statistical mechanics of community detection”. In: *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 74.1, p. 016110.
- Rimkevičienė, J. et al. (2024). “Unravelling the complexity of suicidality: a network analysis of theory-driven and culturally relevant suicide risk factors in a country with high suicide rates”. In: *Nordic Journal of Psychiatry* 78.8, pp. 743–751. DOI: [10.1080/08039488.2024.2424960](https://doi.org/10.1080/08039488.2024.2424960). URL: <https://doi.org/10.1080/08039488.2024.2424960>.

- Rossi, Ryan and Nesreen Ahmed (2015). “The network data repository with interactive graph analytics and visualization”. In: 29.1.
- Shahraeini, Mohammad (2024). “Modified Erdős–Rényi Random Graph Model for Generating Synthetic Power Grids (2024)”. In: *IEEE Systems Journal* 18.1, pp. 96–107. DOI: [10.1109/JSYST.2023.3339664](https://doi.org/10.1109/JSYST.2023.3339664). URL: <https://ieeexplore.ieee.org/document/10364868>.
- Smith, Reginald D. (2002). “Instant Messaging as a Scale-Free Network”. In: *arXiv preprint arXiv:cond-mat/0206378*. DOI: [10.48550/arXiv.cond-mat/0206378](https://doi.org/10.48550/arXiv.cond-mat/0206378). URL: <https://arxiv.org/abs/cond-mat/0206378>.
- Tomassini, Marco, Mario Giacobini, and Christian Darabos (2004). “Evolution of small-world networks of automata for computation”. In: *International Conference on Parallel Problem Solving from Nature*. Springer, pp. 672–681.
- Traag, V. A. (2015). “Faster Unfolding of Communities: Speeding up the Louvain Algorithm”. In: *Physical Review E* 92, p. 032801. DOI: [10.1103/PhysRevE.92.032801](https://doi.org/10.1103/PhysRevE.92.032801).
- Traag, V.A., L. Waltman, and N.J. Van Eck (2019). “From Louvain to Leiden: guaranteeing well-connected communities”. In: *Scientific Reports* 9.1, p. 5233. DOI: [10.1038/s41598-019-41695-z](https://doi.org/10.1038/s41598-019-41695-z). eprint: <https://www.nature.com/articles/s41598-019-41695-z>. URL: <https://doi.org/10.1038/s41598-019-41695-z>.
- Waltman, Ludo and Nees Jan van Eck (2013). “A smart local moving algorithm for large-scale modularity-based community detection”. In: *The European Physical Journal B* 86, p. 471. DOI: [10.1140/epjb/e2013-40829-0](https://doi.org/10.1140/epjb/e2013-40829-0).
- Watts, Duncan J. (1999). *Small Worlds: The Dynamics of Networks between Order and Randomness*. Princeton, NJ: Princeton University Press. DOI: [10.1515/9780691188331](https://doi.org/10.1515/9780691188331). URL: <https://doi.org/10.1515/9780691188331>.
- Watts, Duncan J. and Steven H. Strogatz (1998). “Collective Dynamics of Small-World Networks (1998)”. In: *Nature* 393.6684, pp. 440–442. URL: <https://www.nature.com/articles/30918>.
- Wellman, Barry (1983). “Network Analysis: Some Basic Principles”. In: *Sociological Theory* 1, pp. 155–200. DOI: [10.2307/202050](https://doi.org/10.2307/202050).
- Wierzbinski, Michal, Joaquim Falcó-Roget, and Alessandro Crimi (2023). “Community detection in brain connectomes with hybrid quantum computing”. In: *Scientific Reports* 13, p. 3446. DOI: [10.1038/s41598-023-30579-y](https://doi.org/10.1038/s41598-023-30579-y).
- Wu, J. et al. (2009). “Integrated Network Analysis Platform for Protein-Protein Interactions”. In: *Nature Methods* 6, pp. 75–77. DOI: [10.1038/nmeth.1282](https://doi.org/10.1038/nmeth.1282). URL: <https://doi.org/10.1038/nmeth.1282>.
- Wu, Jianjun et al. (2004). “Urban transit system as a scale-free network”. In: *Modern Physics Letters B* 18.19n20, pp. 1043–1049. DOI: [10.1142/S021798490400758X](https://doi.org/10.1142/S021798490400758X). URL: <https://doi.org/10.1142/S021798490400758X>.
- Yang, Jaewon, Julian McAuley, and Jure Leskovec (2014). “Detecting cohesive and 2-mode communities in directed and undirected networks”. In: pp. 323–332.

- Zhang, Yueheng (2020). *Approximating Graph Conductance: From Global to Local*. Accessed: 2025-10-01. URL: <https://math.uchicago.edu/~may/REU2020/REUPapers/Zhang,Yueheng.pdf>.
- Zhu, Qi et al. (2023). “Multi-Modal Non-Euclidean Brain Network Analysis With Community Detection and Convolutional Autoencoder”. In: *IEEE Transactions on Emerging Topics in Computational Intelligence* 7.2, pp. 436–446. DOI: [10.1109/TETCI.2022.3171855](https://doi.org/10.1109/TETCI.2022.3171855).